

PPT-LAT-Systems

Shannon-Prime Lattice: Systems Architecture

A practical paper for the shannon-prime-lattice project. Restates engine backends, inline compression, model-family coverage, gated sieve/ARM/DHT features, and a formal blockchain design under one umbrella. Mathematical underpinnings are referenced but not derived; this paper is about the system you build and ship.

Abstract

Shannon-Prime Lattice is an inference and coordination substrate. It runs large transformer language models on hardware ranging from commodity desktops to Snapdragon-class phones, compresses the things that make inference expensive (weights, KV cache, gradients), and optionally federates participating nodes into a global compute fabric whose contributions are accounted for on-chain. The mathematical core has been argued elsewhere: the same lattice object underlies token positions, attention scores, weight quantization, KV-cache compression, sieve dominance, and sharded inference. The job of this document is the *systems* story. It describes four engine backends (CPU, CUDA, Vulkan, Hexagon), the inline compression machinery (Q8 weights, Q4 mixed-precision, VHT2 KV cache), model-family support (Llama, Qwen3-series, Gemma 2.5/3/4, DeepSeek V4), the *gated* Lattice features (sieve, ARM, CRT sharding, DHT, token economy), and finally the blockchain design that gives federated participation its accounting layer. Throughout, one invariant is binding: every Lattice-only feature must be off-by-default and the unfeatured baseline must be bit-identical to a plain inference path. The blockchain section is presented as scaffolding, not specification — it is the part of the design we expect to revise most often as constants meet adversarial reality.

Overview: six layers of one math object

A working mental picture of the system: there is one mathematical object — a discrete lattice over a cyclotomic ring, with a CRT decomposition into two coprime Proth primes and an NTT diagonalization that turns convolution into pointwise product — and that single object reappears six times as one walks up from silicon to network.

- At the silicon layer, it is the choice of vector width and integer precision. The NTT primes are sized so that pointwise multiplication fits inside whatever the underlying ALU offers: 32-bit on CPU, 30-bit halves on Hexagon HVX.
- At the kernel layer, it is the matmul: weight tiles in packed integer storage, activations in fp32 or bf16, an inline dequant step that consumes the Frobenius row scale before accumulating.
- At the cache layer, it is the KV representation: K and V are projected onto lattice anchors, stored as small integer blobs, and decoded only when read.
- At the model layer, it is the attention computation: queries and keys live in the same ring, so dot products become polynomial multiplications and we can use cached transforms.
- At the sieve layer, it is the dominance partial order: cached cells form a poset under componentwise lattice comparison, and the engine keeps the dominance-incomparable frontier.
- At the network layer, it is the routing key space: peers hash into slabs indexed by prime factorization, and shards of inference flow over a DHT whose addresses are residues of the same primes that ground the NTT.

The point is not that each layer is metaphysically the same — it is that we exploit the structure of the lattice consistently, so a careful implementation at one layer composes correctly with the layer above and below. The system gets simpler as you understand it more, not more complex.

Section 1: Engine backends

The engine is portable. The same forward pass runs on four backends, dispatched by a thin runtime selector. Each backend implements the same internal kernel surface: a packed-weight matmul, a fused softmax-attention, an FFN with SwiGLU, a positional-encoding op, and the NTT primitive used by attention and (optionally) by sieve operations. Loaders and tokenizers are shared.

1.1 CPU backend

The CPU backend is the reference implementation and the production target for commodity desktops and laptops. It builds on Windows with MSVC (VS2019 Build Tools, MSVC v142 toolset) and on Linux with gcc 11+ or clang 14+. There are two vectorized paths: AVX2 (8-wide single precision, available on essentially every x86-64 chip from the last decade) and AVX-512 (16-wide single precision, plus VNNI for INT8 dot products on capable Intel and AMD Zen 4+ parts). The scalar path remains as a correctness baseline; it is what unit tests check against and what the CRT NTT branches use as an alternate residue to detect arithmetic forgery on the chain.

For matmul, the CPU backend operates row-major-by-token on the activation side and column-major-by-token on the cache side. This asymmetry came out of profiling and is load-bearing: it lets attention prefill batch tokens contiguously while keeping the on-write transposition of K and V cheap. AVX-512 matmul tiles 16 output channels at a time, prefetching the next packed weight tile while accumulating into a register file held in zmm registers. The AVX2 variant tiles 8 channels at a time and uses a software-pipelined prefetch lag of two tiles. Both paths handle Q8 dequant inline: a packed byte arrives, is sign-extended to int16, multiplied by the row's Frobenius scale (a single fp32 broadcast), and added to the running fp32 accumulator. There is no decode arena and no intermediate buffer; the bytes are read directly off the packed weight storage.

For attention, the CPU backend has two modes. In dense mode it computes the full QK^T softmax V chain at fp32. In poly-ring mode (default at large head dimensions) it transforms Q and K into the NTT domain, computes pointwise products there, and inverse-transforms only the output. The NTT itself uses Barrett reduction with precomputed mu constants; we measured 4x kernel speedup from this versus a naive Montgomery layout. The K cache holds its NTT-domain transform persistently across forward steps, so generation never re-transforms a key that has already been seen.

For FFN, SwiGLU is fused: the gate projection, up projection, silu, and elementwise multiply happen in a single pass over the hidden state, so the activation never has to round-trip through DRAM between sub-ops. This is meaningfully faster on memory-bandwidth-bound CPUs.

Threading is via standard pthreads or std::thread; the engine's job pool is sized to one worker per physical core minus one (the conductor thread). NUMA-aware placement is supported but off by default; on dual-socket servers, enabling it gives roughly 1.4x on prefill but introduces locality bugs we are still characterizing.

A few additional CPU-backend details that matter to operators. First, the build is statically linked by default — we don't depend on any shared library at runtime beyond the C runtime and a small portable threading shim. This avoids the usual library-versioning hell on Linux

distributions and on stock Windows installs without redistributables. Second, the binary is single-file: the model is a separate file (the GGUF), the engine is one executable, and the configuration is environment variables and command-line flags. There is no install step. Third, the engine respects two affinity hints — `SP_CPU_CORES=N` to cap the worker pool at N workers, and `SP_CPU_AFFINITY=mask` to pin workers to a specific core mask. These are operator escape hatches; the auto-detected defaults are correct for almost every machine.

One subtle behavior worth calling out: the CPU backend will silently fall back from AVX-512 to AVX2 if it detects an AVX-512 frequency-throttling event on Intel Skylake-X and Ice Lake parts. The detection is heuristic — we measure the wall-clock cost of a fixed-size matmul during warmup and compare AVX2 versus AVX-512; if AVX-512 is slower by more than 10% we stay on AVX2 for the rest of the session. This came out of bench results where AVX-512 should have been faster on paper but was slower in practice because of the throttling. The behavior can be disabled with `SP_CPU_FORCE_AVX512=1` for the benchmarker who knows what they want.

1.2 CUDA backend

The CUDA backend targets RTX 3000 and 4000 series consumer cards. It builds with NVCC + MSVC + CUDA 12.4, currently against compute capability 8.6 (Ampere) and 8.9 (Ada Lovelace). The build system requires Visual Studio 2019 Build Tools and the `--use-local-env` flag through CMake; this is brittle but documented in the repo, and shaves roughly five minutes off the configure step compared to the recommended path.

Kernels are implemented in CUDA C++. Matmul uses warp-cooperative tiles: each warp owns a 16x16 output block, with shared-memory tiles for the weight side and register tiles for the activation side. We do not currently use Tensor Cores; the Q8 dequant path is not yet expressed in a way that maps to `mma.sync`, and the gain from converting it would be modest until we also push activations into bf16, which is a separate workstream. NTT is implemented as a Cooley-Tukey radix-2 butterfly with the twiddle factors held in constant memory; for the configured $N=256$, the entire butterfly fits in a single CTA and the NTT cost is dominated by the memcopy in/out, which we plan to eliminate by fusing NTT into the K cache writeback.

Attention has two implementations: a fused-softmax dense kernel and a poly-ring kernel. The fused-softmax kernel is competitive with FlashAttention v2 at sequence lengths up to about 4K on Ampere; beyond that, FlashAttention's online softmax becomes important and we lose ground. We have a port of online softmax in flight but have not validated it for arithmetic parity with the CPU reference yet, which is required before it can be used by chain validators.

Falls back to CPU for unsupported ops, which currently includes the sieve dominance check (it's a CPU op anyway; lattice comparisons branch too much to be GPU-friendly) and the freethedsp/Hexagon-specific code paths (obviously). The fallback is automatic and silent; a debug flag (`SP_DEBUG_FALLBACK=1`) prints when it engages.

Multi-GPU is supported via a tensor-parallel layout across NVLink-connected cards. The simplest form, two-way TP, splits each attention head's QKV across two GPUs and the FFN intermediate dimension across two GPUs; the all-reduce happens at the residual-stream junction. We have validated this on a pair of RTX 4090s for Llama 3.1 8B and Qwen3 7B and seen close to linear scaling on prefill, with generation showing the expected NCCL latency tax. Four-way and eight-way TP are coded but not validated on consumer hardware (we lack the boxes); they should work on data-center cards. Pipeline parallelism is not implemented and not planned in the immediate term; we prefer the simplicity of pure TP at the scales we target.

GPU memory budgeting matters because consumer cards are tight. For Llama 3.1 8B at Q8 with 32K context and KV compression on, we need roughly 9 GB on a single RTX 3090/4090 (24 GB), which leaves plenty of headroom. For Llama 3.1 70B at Q8 we cannot fit a single card; two-way TP across two 24 GB cards is the minimum, and four-way is comfortable. For the 671B

DeepSeek V4, no consumer card configuration fits the model resident; expert streaming from NVMe is mandatory, and a fast PCIe Gen 4 NVMe is the binding constraint on per-token latency.

1.3 Vulkan backend

The Vulkan backend exists to reach AMD and Intel GPUs and to give us a portable GPU baseline that does not require CUDA. It builds against the Vulkan SDK 1.3.x and uses glslc for shader compilation. Compute shaders implement matmul, NTT butterflies, attention, and FFN; the same SPIR-V binaries run on Windows, Linux, and (in principle) Android, though the Android path is not yet wired into the production loader.

Vulkan is a lower priority than CUDA today because we do not have validated production results on AMD parts at the Llama-3 scale, and because the shader-compilation step adds a measurable cold-start cost (several seconds on first run; cached after that). However it is the only viable cross-platform GPU path, and it is what will eventually run on Intel Arc, Apple silicon (via MoltenVK), and Adreno mobile GPUs. The engine treats Vulkan symmetrically with CUDA at the dispatch level: same kernel surface, same fallback semantics.

One specific Vulkan caveat: subgroup operations are not uniformly supported across vendors. We currently use only the subgroup ballot and shuffle intrinsics that are guaranteed by the Vulkan 1.2 core spec, which forces some matmul reductions onto shared-memory paths that would be cheaper as warp shuffles in CUDA. The gap closes once we are willing to require Vulkan 1.3 device extensions for AMD RDNA3 and Intel Xe2; we have not made that call yet.

1.4 Hexagon backend

The Hexagon backend is the phone path. It targets the Snapdragon 8 Gen 1's V69 Hexagon Tensor Accelerator, accessed via FastRPC from the Android ARM cores. The build environment is the Hexagon SDK 5.x on a Windows host, with the specific patch documented in the build recipe (the SDK's hexagon_fun.cmake points at bin/qaic, but on Windows the binary is WinNT/qaic.exe, and the Git sh.exe has to be prepended to PATH so the shell wrappers resolve correctly).

The backend has three logical layers, used in different combinations depending on what the device exposes:

- **Halide-compiled HVX kernels.** These are the hand-rolled vector kernels: matmul, Q8 dequant, NTT, attention. HVX is 128-byte vectors operating on fixed-point INT8/INT16 lanes; the matmul kernel produces 32 output channels per HVX vector load. Halide is the source language; it is the same code that has been validated on the S22U at 81 us minimum dispatch time, which matches the AI Hub reference run for our test model.
- **QNN HTP backend.** QNN (Qualcomm Neural Network SDK) lets us schedule operations on the V69 HTP using either AOT-compiled .bin graphs or runtime-built graphs constructed via QnnGraph_addNode. The runtime path is the production target because it lets us reconfigure the matmul shape per-layer without a recompile. The dispatch ceiling we measured is roughly 577 calls/sec over FastRPC; this is the hard upper bound on per-token throughput at fine granularity. Our 4-split-per-token layout sits comfortably under that ceiling at 38.7 t/s.
- **freethedsp shim.** Production-locked Android devices restrict signed-PD attachment to apps with the right certificates. The freethedsp shim, adapted from public work in the community, opens the DSP via an unsigned PD by LD_PRELOAD'ing into the FastRPC client. It is opt-in via SP_FREETHEDSP=1; the default path uses signed PD where available. The shim is the only way to bench on a stock retail device without unlocking the bootloader.

Mode C of the Hexagon backend (QNN HTP for matmul, HVX for everything else) is the current production target. Mode D (the ISP-as-spectral-reconstructor pipeline, where the Spectra 680

image-signal-processor performs skeleton+residual band fusion at 18-bit fixed point in parallel with HTP matmul) is research; the pieces have been validated individually but not strung together.

FastRPC scratch buffer allocation has one footgun we hit repeatedly: the rpcmem registration size must exactly equal the IDL length parameter. Over-allocation silently fails with AEE_EUNSUPPORTED, which looks like a totally unrelated DSP error. This is documented in the repo and worth repeating here because every new contributor hits it.

A second Hexagon-specific note: thermal management. The V69 HTP can run at full power for short bursts but throttles after sustained load — typically after 30-60 seconds of continuous inference on a phone with passive cooling. The throttle drops throughput by roughly 40%. Our default scheduler accommodates this by inserting micro-pauses (1-2 ms) between layers when the host thermal sensor reports the SoC entering a warning zone. The pauses are cheap on the per-token critical path but visibly reduce throughput; an operator who needs sustained peak performance on a phone needs an external cooling solution, which is mostly a research-rig configuration. Production usage on a normal phone is bursty (a few seconds of inference followed by user reading time) and rarely hits the throttle.

Memory on the phone is the other binding constraint. On the S22U test rig (12 GB total system RAM, of which roughly 8 GB is available to user processes), we can fit a Qwen3-3B at Q4 with the full Lattice cache enabled. The 7B class is uncomfortable; 8B requires aggressive Q4 across all layers and trims context to 8K. Larger phone SoCs (24 GB on the upcoming flagship class) loosen this enough that 7B-class models become comfortable; we benched a preview device and saw the expected linear scaling. The Hexagon backend is therefore well-positioned for the model-size class that will dominate consumer use for the next 18 months, which is roughly 1B-7B.

1.5 Common loader and dispatcher

GGUF is the on-disk model format. The loader is shared across all four backends; it memory-maps the file, validates the header, walks the tensor table, and produces a uniform tensor descriptor that the backend-specific dispatcher consumes. The descriptor carries dtype, shape, role (e.g., “attn.wq.layer_5”), and any quantization metadata (Frobenius scale for Q8, calibration constants for Q4). Loader extensions are how new model architectures are added: a new arch declares its tensor inventory and the per-tensor roles, and the dispatcher routes ops accordingly.

The dispatcher itself is a small function-pointer table per backend. At engine init, the selected backend populates the table; the forward pass then calls through the table without any per-op branching. This is what makes backend-swap cheap: there is no scattered if (backend == CUDA) logic anywhere in the model code.

Backend	Build env	Targets	Matmul	NTT	Attention	KV compress
CPU AVX2	MSVC / gcc	x86-64 universal	yes	scalar	dense + poly	yes
CPU AVX-512	MSVC / gcc	Skylake-X, Zen 4+	yes	Barrett SIMD	dense + poly	yes
CUDA	NVCC + MSVC, CUDA 12.4	Ampere, Ada	yes	radix-2 butterfly	fused + poly	partial
Vulkan	Vulkan SDK 1.3 + glslc	AMD, Intel, mobile	yes	butterfly shader	fused	partial

Backend	Build env	Targets	Matmul	NTT	Attention	KV compress
Hexagon HVX	Hexagon SDK 5.x	V69 HTP (SD8G1)	yes	INT16 fixed	poly	yes

1.6 Polynomial-Shift Relative Attention

Standard Rotary Positional Embeddings (RoPE) compute relative phase shifts using continuous floating-point trigonometry — a $\cos / \sin$ table of size $O(\text{ctx} \cdot D)$ that becomes painful at long context. The engine collapses this by routing relative attention through the negacyclic cyclotomic ring $R_q = \mathbb{Z}_q[x]/(x^N + 1)$ that already underlies the NTT-attention path.

The mechanism. Inside R_q , multiplication by $e^{i \cdot \Delta \cdot \theta_d}$ is represented as a *polynomial index shift modulo N* — a discrete integer permutation of the coefficient vector. The continuous trigonometric rotation is replaced by exact arithmetic on residue indices. The relative-attention rotation cache therefore stores **one int32 shift offset per token** rather than D fp32 values per token.

For $D = 128$, $\text{ctx} = 4096$:

- Standard fp32 rotation cache: $4096 \times 128 = 524,288$ floats = 2 MB
- Polynomial-shift cache: 4096 int32 offsets = 16 KB

~128× memory reduction, mathematically lossless on stock geometric-RoPE models.

The underlying $\theta_d = \text{base}^{-2d/D}$ schedule does not need to change — the ring representation turns every rotation into a discrete index shift regardless of how the frequencies are spaced. There is no ϕ -RoPE precondition, no frequency-axis sort, no Three-Gap dependency. The win is structural to the cyclotomic ring, not to the frequency schedule.

Why this is lossless. The NTT-attention path already gates on T_PR_2 / E_CPU_5 — the negacyclic polynomial multiplication is exact arithmetic in the ring up to the int32 quantisation of the head vectors. The rotation-as-shift representation introduces no error beyond that quantisation; once E_CPU_5 measures $\text{KL} < 10^{-7}$ against the fp32 reference (we have it at $\approx 2.7 \times 10^{-10}$ mean on Qwen3-0.6B), the polynomial-shift cache is in-spec by construction. Activating it does not change the gate.

Gating. The polynomial-shift cache is coupled to SP_ENGINE_NTT_ATTN=1. When NTT-attention is active, the rotation cache is the integer-offset table; when NTT-attention is off, the engine falls back to the standard fp32 $\cos / \sin$ table. There is no separate SP_ROPE_* flag.

Backend impact. All four backends benefit; the win is largest on CPU (where transcendentals are slow) and Hexagon (where they are slow *and* memory-bandwidth-bound). CUDA and Vulkan benefit modestly — the rotation tables were already in shared memory there — but the cache footprint reduction translates directly into more room for the KV cache at long context.

What this section used to claim, and what changed. Earlier drafts proposed a “Three-Step Relative Attention Cache” that derived a ~43× collapse from the Three-Gap Theorem (T7). That mechanism requires the model’s RoPE frequencies to be *linear* multiples of an irrational, but stock RoPE is *geometric* and does not satisfy the precondition. The Phase 2-CPU agent flagged this on 2026-05-22. The fix is the rewrite above: instead of trying to swap the model’s positional basis to make Three-Gap apply, route relative attention through the ring representation that the framework already commits to. The win comes out at ~128× rather than ~43×, applies to every stock model without retraining, and uses a kernel (NTT-attention) that has already been measured at $\text{KL} < 10^{-9}$.

The original ϕ -RoPE schedule swap and Three-Gap frequency-sort restructuring are kept as research-track items in PPT-LAT-Roadmap §20 for any future model that ships with linear-in- ϕ RoPE.

Section 2: Inline weight and cache compression

Compression is foundational, not an option. Everything else assumes weights are packed and the KV cache is encoded; the unpacked path exists for validation and is not optimized.

2.1 Q8 weight storage with Frobenius scale

Each weight tensor is stored as a stream of packed bytes plus a per-row scale array. The packing is straightforward: each fp32 row is normalized by its largest absolute value, scaled into the $[-127, 127]$ range, rounded with ceiling-shift to avoid round-half-up bias, and stored as signed int8. The scale (a single fp32 per row) lives alongside the packed bytes.

Decompression is inlined into matmul. For each output element, the kernel reads the corresponding packed weight byte, sign-extends to int16, multiplies by the row's scale (broadcast to fp32), and accumulates into the output. On AVX-512, this is one zmm load of 64 packed bytes, one sign-extend to two zmm registers of int16, two multiplies against a broadcast fp32 scale, and an accumulate. On HVX, it is one vector load of 128 packed bytes, two vector multiplies against a precomputed INT16 scale, and an accumulate. There is no intermediate decode arena; the packed bytes flow directly from DRAM into ALU lanes.

The memory saving is 4x versus fp16 and 8x versus fp32. On the Gemma3-1B test rig, this brought a 10.4 GB weight footprint down to 1.3 GB, which matters because it leaves headroom for KV cache, scratch buffers, and the lattice sieve on devices where DRAM is the binding constraint. The accuracy cost is small: production logs show Q8 perplexity within 0.6% of the fp16 baseline on a calibrated chunk-4 evaluation. The cost is small enough that we ship Q8 as the default and treat fp16 as a debugging mode.

There is a background prefetcher path for Q8 weights: a dedicated thread maintains a two-slot double buffer ahead of the forward pass, with the forward path acquiring a layer at a time via condvar handoff. The architecture is in production but the speedup is not yet visible at Gemma3-1B scale because of memory-bandwidth contention between prefetcher writes and matmul reads. Larger models, where the per-layer compute time grows faster than the per-layer fetch time, are where this pays off, and that work is queued.

2.2 Q4 mixed-precision path

Q4 is the same architecture as Q8 but with 4-bit packed nibbles, two per byte. The dequant arithmetic is identical: nibble, sign-extend, multiply by row scale, accumulate. The packing density is 16x over fp32 and 8x over fp16, which makes it possible to fit large dense models on phone-class devices.

The catch is accuracy. A first-pass Q4 implementation with a single per-tensor scale produced catastrophic signal collapse on the same Gemma3-1B test rig (perplexity blew up by nine orders of magnitude). The cause was the choice of scale granularity: Q4 has so few discrete levels that a tensor-wide scale clips meaningful variation. The current path is per-row scales (one fp32 per row), the same as Q8, with a calibration pass that selects a clipping percentile rather than the row max. Under this scheme, early bench shows perplexity within roughly 3% of fp16, but the validation work to make this production-ready is not done.

Q4 is therefore opt-in via `--frobenius-q4` and is not the default. It will become default when (a) we have a calibration sweep across all supported model families showing PPL within 1% of fp16 on calibration data and (b) we have a backstop calibration-failure detector that refuses to load a Q4 model that didn't calibrate well.

Mixed precision means the heaviest layers (attention output proj, FFN gate/up/down) stay Q8 while the dominant memory cost (the wq/wk/wv stack and the embedding) goes Q4. The mix is selected per arch by a config file; for Llama-3.2 we mix to roughly 5-bit average; for Qwen3-MoE the expert weights are aggressively Q4 while the router stays Q8 because router miscalibration tanks expert routing quality.

2.3 Inline KV cache compression (VHT2 + Spinor)

KV cache is the second large memory cost after weights, and at long context it dominates. The Lattice cache uses a three-stage encoder:

1. **VHT2 projection.** Each new K vector is projected against a fixed set of lattice anchors. The projection is computed in fp32 and quantized to a small integer per anchor.
2. **Mobius mixing.** A short mixing step rebalances anchor mass across siblings on the lattice tree, reducing the worst-case quantization error.
3. **Spinor packing.** The final representation is a 63-byte Spinor block per cache cell: a small header (norm, exponent), the anchor coefficients (packed integers), and a checksum byte. 63 bytes per cell against the roughly 8 KB an uncompressed fp16 cell would take at $d=4096$ is the roughly 130x cache compression we cite in benchmarks.

Decoding happens on read, inside the attention kernel. The dequant logic is essentially the same as Q8 weights — read packed integers, multiply by anchor weights, accumulate — except the anchor table is small and shared across all cache cells, so it fits in L1 on the CPU and in shared memory on the GPU.

The accuracy story for KV compression is harder than for weights. Cache state accumulates errors across timesteps, so even a modest per-cell quantization error compounds in long-context settings. Our mitigation is a periodic *refresh*: every K tokens, the cache cells for tokens older than K are recompressed with a slightly larger Spinor block (96 bytes instead of 63), trading some compression for stability. The refresh interval is tunable; the default of $K=512$ keeps perplexity drift under 1% out to 32K context on the models we've tested.

The KV compression path is engaged by default at long context ($n_{ctx} > 2048$) and bypassed for short prompts where the savings aren't worth the encode overhead.

A further question we get often: why not just use one of the existing quantized KV approaches (KIVI, KVQuant, the GPTQ-K family)? The honest answer is that we evaluated them and found that they trade compression ratio for either accuracy or compute cost in ways that don't suit our workload. KIVI is comparable to our scheme at 4-bit and is simpler to integrate; but it doesn't compose with the lattice sieve, because its cell representation is not a dominance-comparable structure. KVQuant's per-channel scales are competitive but require a calibration pass we don't always have time to run for a freshly-loaded model. Our VHT2+Spinor encoder is the design that gives us the dominance comparability the sieve needs, with a compression ratio in the same neighborhood as the published alternatives. The cost is a more complex encoder; we believe that cost is justified by the downstream gating opportunities the sieve enables. If we are wrong about the value of the sieve, the KV encoder is the easiest part to swap; the abstraction boundary between the encoder and the rest of the system is clean.

Fibonacci sub-sampling for KV eviction. When the context window exceeds memory bounds, the engine employs Fibonacci sub-sampling for KV eviction instead of FIFO or LRU. Standard eviction schemes degrade long-range attention resolution because they bias toward retaining either the oldest or the most-recently-used tokens, leaving gaps in the middle of the

context where retrieval queries often need to land. By Theorem T7 (PPT-LAT-Theory), retaining tokens at positions $\lfloor k\varphi \cdot N \rfloor \pmod N$ where $\varphi = (\sqrt{5} - 1)/2$ guarantees the retained cache maximally and uniformly covers the temporal context regardless of N , with bounded discrepancy on the temporal axis to mirror the frequency-axis equidistribution exploited by E9.1. Engaged via SP_KV_FIB=1; default OFF until the long-context PPL-drift sweep validates parity with the existing refresh-window scheme.

2.4 Per-hardware code paths

Different vector widths and dequant intrinsics matter enough that we keep separate code paths for each:

AVX2. 8-wide fp32 matmul, integer Q8 dequant via sign-extend and broadcast scale. Compatible across the entire Haswell-and-later x86 world. The fallback when AVX-512 isn't available.

AVX-512. 16-wide fp32 matmul. Uses VNNI (vpdpbusd) for INT8 dot products when available, falling back to a manual vpmaddubsw + vpaddw chain when VNNI isn't. The BF16 fast path is also AVX-512-only; we use it for KV cache value side, where the precision loss versus fp16 doesn't materially affect attention quality.

HVX (Hexagon). 128-byte vectors, fixed-point INT8/INT16 lanes. The HVX matmul kernel operates on 32 output channels per HVX vector and accumulates into a wide register pair. Q8 dequant on HVX uses the Vh_vmpyio_VhRh pattern for the int16 multiply against a broadcast scale.

DSP scalar. Fallback for ops that don't have a vector form on the DSP (rare in production; most ops vectorize). The scalar path is also what we use for low-volume control logic like the per-layer norm.

ARM NEON. Used on the phone's ARM cores for ops that don't go to the DSP — tokenization, sampling, the host-side glue around FastRPC calls. NEON Q8 dequant uses the standard vmovl_s8 + vmulq_f32 pattern; we don't currently use Apple-specific instructions, which means the same code runs on Snapdragon and on Apple silicon via Rosetta translation when needed.

Section 3: Model-family support

Each model family is a forward-pass dispatcher plus a loader extension plus a quantization-sweep validation. The engine's job is to provide the kernels; the family-specific job is to wire them up correctly.

3.1 Llama

Llama is the reference architecture. The engine currently supports Llama 3.1 and 3.2 in production. The forward pass is the standard transformer: RMSNorm, RoPE, grouped-query attention with 8:1 group ratio in the larger sizes, SwiGLU FFN. RoPE is implemented with precomputed sin/cos tables; the GQA cache layout uses a shared K/V slab per group rather than duplicating, which gives the expected 8x cache savings at the 8B size.

Llama 3.2 introduced a tied embedding for the 1B/3B sizes; the loader detects this and reuses the embedding tensor for both input embed and LM head. This is small but easy to get wrong: the LM head still needs the final RMSNorm before projection, even when the projection matrix is the embedding.

The Q8 sweep on Llama 3.1 8B shows PPL 7.18 vs an fp16 baseline of 7.13 (+0.7%), within the 1% bar. The Q4 sweep is not done yet for 3.1; for 3.2 1B we have Q4 at PPL 13.42 vs fp16 13.05 (+2.8%), which is over our bar but not catastrophic. Reasoning: 1B is small enough that Q4 quantization noise is a significant fraction of the model's total expressive capacity. The same arch at 8B is expected to handle Q4 better, but the sweep is queued.

3.2 Qwen3 series

The Qwen3 family is the SOTA target family. Production support exists for:

- **Qwen3** (0.5B, 1.5B, 3B dense). Standard transformer; the main quirk is the tokenizer (BBPE with a specific Chinese-heavy pretraining vocabulary that affects byte-fallback behavior). Q8 PPL within 0.5% of fp16; Q4 within 2% at 3B size.
- **Qwen3.5** (3B, 7B dense). Identical to 3 architecturally; the loader treats them as one family with version-tagged norm constants. The pretraining diet shifted toward more code, which means Qwen3.5-Coder is a particularly strong target for spec-decode (small draft model + bigger verifier).
- **Qwen3.6 MoE**. This is where the family diverges. MoE routing adds an expert-selection gate that runs once per token per layer; the engine implements it as a top-K softmax against the router weights, with K=2 in production. Each expert is a separate FFN slab in the GGUF; the loader walks all of them and packs them as a contiguous Q4 expert pool with a shared scale array. The dispatcher's FFN path becomes a switch: route to expert(s) by the gate's top-K, run the FFN on those experts, combine outputs weighted by gate probabilities. Production Q8 PPL on Qwen3.6-30B-A3B sits within 1.2% of fp16 — a slightly larger gap than dense models because router miscalibration under quantization is a meaningful source of error.
- **Qwen3.7** (announced; not yet supported). The architectural disclosures so far suggest a hybrid SSM+MoE block (similar in spirit to Qwen3-Next), gated attention, and an mRoPE mode 8. Loader extension scope is small but not yet done.

The Qwen series is also where we have the best spec-decode results: a 0.5B draft model running ahead of a 3B verifier hits 2.16x spec-decode speedup, and stacking with the Hexagon fused KQ path on the verifier brings that to 2.23x. Spec-decode interacts with KV compression in a subtle way: the draft and the verifier must agree bit-exactly on the verified prefix, which means their KV caches must use the same encoder configuration. We enforce this by requiring both models to be loaded with the same `SP_LATTICE_SIEVE` value.

3.3 Gemma 2.5/3/4

Gemma's signature is its attention pattern: a sliding-window attention interleaved with full attention, in a 5:1 local-to-global ratio in Gemma 3. This means most layers see only the last W=4096 tokens, while every sixth layer attends to the full context. The dispatcher's attention path consults a per-layer flag that selects between the local and global kernel. Both use the same poly-ring attention math; the only difference is the SWA mask bounds.

Gemma 2.5 adds post-attention and post-FFN norms — RMSNorm not just on the residual input but on the residual output. The loader sets up four norm slots per block instead of two; the dispatcher's block path runs them at the appropriate points. The cost is small but the order matters; a wrong-order norm produces a model that is coherent-sounding but measurably worse on PPL benchmarks, which makes the bug easy to miss without a proper sweep.

Gemma 3 added the local/global pattern. Gemma 4 (in preview) reportedly extends the SWA window and adds a routing-style gate to the FFN, but we have not yet seen a checkpoint we can load.

Q8 PPL on Gemma 3 4B sits within 0.4% of fp16. The Gemma family benefits unusually well

from KV compression because the SWA window already bounds the working cache size; the Lattice cache then compresses *within* the window, so per-token cache cost drops by another order of magnitude.

3.4 DeepSeek V4

DeepSeek V4 is the deep end. 671B parameters MoE with roughly 37B active per token. FP8 training native, which means the published checkpoints are FP8 from the start; converting to fp16 for our standard pipeline doubles the disk footprint. Multi-token prediction (MTP) lets each forward pass produce K candidate next tokens that the model itself can verify; this is a form of self-spec-decode that interacts cleanly with our Lattice cache.

The DeepSeek MoE routing is more complex than Qwen’s. It uses a learned load-balancing loss during training that biases the gate toward a balanced expert assignment; at inference, we still take top-2, but the gate has more dynamic range, which means the top-2 weights are closer to (0.5, 0.5) than to (0.8, 0.2). This matters for quantization: the secondary expert contributes meaningfully to the output, so Q4 on experts is more sensitive than on Qwen3.6.

The engine’s DeepSeek V4 support is partial. We can load and run the model on the CUDA backend at full size, with experts streamed from disk because they don’t fit in any commodity GPU’s VRAM. Streamed-expert inference is slow — a single forward pass is dominated by SSD bandwidth, not compute — but it works, and it is the prerequisite for the CRT-sharded multi-node inference path described later. The Hexagon backend cannot host DeepSeek V4 at all; the model is simply too large for any phone-class device. The Vulkan backend is gated on the streamed-expert infrastructure being ported to its dispatcher, which is on the roadmap.

DeepSeek V4 is also our test case for the most aggressive compression configurations: Q4 experts, Q8 attention, BF16 KV cache. We have an internal eval running showing PPL within 2% of FP8 baseline on a subset of HumanEval and MMLU; the full sweep is pending. This is the model where the next year of optimization will pay the most attention.

Family	Sizes	Backends	Q8 PPL gap	Q4 PPL gap	Notes
Llama 3.1	8B, 70B	CPU, CUDA	+0.7%	n/a yet	reference arch
Llama 3.2	1B, 3B	all four	+0.5%	+2.8% (1B)	tied embed
Qwen3	0.5B-3B	all four	+0.5%	+2%	best draft for spec
Qwen3.5	3B, 7B	CPU, CUDA, Vulkan	+0.6%	+2.1%	code-focused
Qwen3.6 MoE	30B-A3B	CPU, CUDA	+1.2%	sensitive	top-2 routing
Qwen3.7	tba	unsupported	-	-	hybrid SSM+MoE
Gemma 2.5	2B, 9B	CPU, CUDA	+0.4%	+1.8%	post-attn/FFN norms
Gemma 3	1B, 4B, 12B, 27B	all four	+0.4%	+1.5%	SWA 5:1
Gemma 4	tba	unsupported	-	-	preview
DeepSeek V4	671B-A37B	CUDA (streamed)	+2% (partial)	sensitive	FP8 native, MTP

Section 4: Sieve / Lattice features — gated

This is the part of the system where the engine becomes Lattice. Up to this point, everything described is a fast, compressed, portable inference engine. The features in this section turn it into a node in a federated fabric. They are all gated, off by default, and bypassed entirely in the baseline path. The *regression invariant* is binding: if every gate is off, the engine produces output that is bit-identical to a plain inference path. We test this every release with a parity sweep across the four backends.

4.1 KSTE-encoded KV cache (SP_LATTICE_SIEVE)

KSTE (Key-Side Tree Encoding) takes a KV cache cell and emits a small structured tree: the root holds a coarse fingerprint of the cell, internal nodes hold residual error against the parent fingerprint, and leaves hold the Spinor block. The tree representation has two purposes. First, it makes dominance comparison cheap: two cells can be ordered by their tree fingerprints without decoding the leaves. Second, it makes deduplication via the Friedman sieve straightforward: the sieve keeps only the dominance-incomparable trees, dropping cells whose trees are strictly dominated by another cell already in the cache.

The Friedman sieve runs on cache write. When a new cell arrives, the sieve walks the existing frontier (typically a few hundred entries deep) and checks whether the new cell dominates or is dominated by any frontier entry. Dominated cells are dropped; dominating cells push out the existing entries they dominate. The complexity is $O(\text{frontier size})$ per insert, which is acceptable up to frontiers of a few thousand cells, beyond which we keep a coarser top-level index by slab.

With the gate off, KV writes go straight into the standard VHT2+Spinor encoder; with the gate on, they additionally pass through the KSTE encoder and the sieve. The bit-identity property is maintained because the standard decoder doesn't consult the KSTE structure; it reads the Spinor block, which is unchanged.

4.2 ARM aggregation (SP_LATTICE_ARM)

ARM (Algebraic Resonance Memory) is an HRR-style associative memory living in the cyclotomic ring. Each bound key-value pair (k, v) contributes a single bound element in the ring; multiple bindings accumulate via ring addition, producing a slab whose state encodes the aggregate of all bound pairs. Recall on a query key q produces a noisy approximation of the bound value, with noise scaling as $1/\sqrt{K}$ for K bindings.

At a single node, ARM is a side-effect: bindings happen on chunk boundaries during forward inference, and recalls happen as an injection in the attention path. With the gate off, neither happens. With the gate on, ARM is *local-only* by default — bindings stay on the local node — and the contribution to forward inference is a small alpha-weighted bias on the attention scores.

Multi-node ARM is where the federated story begins. Slabs are gossiped between peers; a node receiving a slab adds it to its local bank, so over time every participating node accumulates a shared associative memory of bindings made anywhere in the network. The aggregation is the literal ring sum, which means it commutes and associates cleanly; no consensus is required to merge slabs, only delivery. This makes the ARM gossip protocol simple: each node periodically emits its slab deltas; peers integrate them on receipt.

The capacity-bound mitigation is important here. ARM bindings have a cosine-similarity capacity curve that degrades as K grows. With multi-node aggregation, K can grow without bound, so a participating node must either periodically rebind from a fresh slab or accept that older bindings become noisier. The current design is periodic rebinding: every interval (default 1 hour wall-clock), each node forgets bindings older than a threshold and starts a fresh slab. This bounds the capacity but means short-term gossip is the primary value of ARM, not long-term memory.

Golden-ratio key generation. ARM requires binding keys that are mutually near-orthogonal under circular convolution. The previous design used random projection with a Gram-Schmidt residual to enforce orthogonality; the cost was a heavy initialization pass at slab creation. The new path generates keys deterministically using φ -spaced phases in the cyclotomic ring R_q — specifically, the k -th binding key has phase $2\pi k\varphi \bmod 2\pi$, with the rest of the ring coordinates derived from a fixed seed. By Three-Gap (PPT-LAT-Theory T7), this gives near-orthogonality

structurally with no Gram-Schmidt cost. The empirical capacity curve, currently $0.83 \rightarrow 0.15$ cosine recall across $K = 1, \dots, 64$ sparse bindings, is expected to extend toward $K = 128$ or beyond under φ -spaced initialization; the validation sweep is in Phase 9 of the roadmap.

4.3 CRT-sharded inference (SP_LATTICE_CRT_SHARD)

CRT sharding is a way of running one inference across two nodes by splitting the NTT computation across the two coprime Proth primes q_1 and q_2 . Each shard holds the weights in residues mod q_1 (one node) or q_2 (the other); the activations are sharded the same way. Forward inference proceeds in parallel on both nodes; the outputs are CRT-reconstructed at the very end to produce a single fp32 logit vector for sampling.

The bandwidth requirement between shards is small: only the residue-domain activations need to cross the wire, and those are about 30 bits per scalar (vs 32 for fp32). The latency requirement is harder, because synchronizing twice per layer (once before attention output combine, once before FFN output combine) means the inter-node round-trip lands on the per-token critical path. At 100 ms inter-node latency and a 32-layer model, this adds 6.4 seconds per token, which is a non-starter for interactive use; at 5 ms LAN latency, it adds 320 ms per token, which is workable.

The pragmatic conclusion is that CRT-sharded inference is for batch or research workloads, not for interactive chat, until we have the data center deployment we don't yet have. It is also one of the open questions: there may be a way to relax the per-layer synchronization to per-block (every 4 layers, say) at some cost in numerical accuracy, which would let CRT sharding work over WAN. We have not characterized that tradeoff.

With the gate off, CRT sharding doesn't engage; the engine runs single-node and computes the residues internally for verification. With the gate on, the engine looks for the named peer at the configured address, establishes a session, and switches to the shard protocol.

4.4 DHT participation (SP_LATTICE_DHT)

The DHT (Distributed Hash Table) is what makes the Lattice a network rather than a collection of isolated nodes. It uses a **2-Axis Fibonacci-Prime Address Space** — a hybrid of semantic and load routing that solves the standard Kademlia clustering problem natively without resorting to cryptographic hashing.

Axis 1 — Semantic axis (prime-factored lattice). The prime factorisation of lattice slab indices routes content based on semantic adjacency: related content lives at nearby prime indices, by construction of the KSTE encoder (PPT-LAT-Theory §4). Slabs corresponding to the same prime root are likely to hold similar content; queries against related semantic neighbourhoods naturally cluster their traffic on the same DHT subtree.

Axis 2 — Load axis (Fibonacci hashing). Pure semantic routing would concentrate traffic on popular semantic neighbourhoods, leaving other nodes idle. To prevent this, node addresses *within* a semantic slab are derived by multiplying the node ID by $\varphi = (\sqrt{5} - 1)/2$ and taking the fractional part — the standard Fibonacci hashing of Knuth §6.4, which by Three-Gap (PPT-LAT-Theory T7) maximally distributes load within the slab regardless of the underlying ID distribution.

The two axes compose: the semantic axis decides *which* slab handles a request; the Fibonacci axis decides *which node within the slab* handles it. Because the two axes are mathematically independent, traffic skew on one axis does not propagate to the other — popular semantic neighbourhoods stay popular, but the nodes within them are uniformly loaded.

The DHT carries the same payload types as a single-axis Kademlia variant:

- **Sieve cache deltas.** When a node accepts a new KSTE tree on its local sieve, it gossips the tree to peers whose semantic-slab overlaps. Peers integrate the delta if it adds to their incomparable frontier.
- **ARM slab updates.** Periodically (default once per minute), each node emits its ARM slab delta. Integration is ring addition.
- **Block propagation.** New blocks (see §6) propagate via standard gossip with deduplication by block hash.
- **Sharded inference recruitment.** When a node wants a CRT-sharded inference peer, the DHT lookup runs against compatible shard assignments.

Bootstrap: `SP_LATTICE_DHT=<peer_addr>` points at a known bootstrap node; from there, standard Kademlia bootstrap (k-buckets, refresh) discovers peers along both axes.

This is also the substrate for the crawler: when the lattice sieve identifies a slab whose coverage is thin, the crawler requests dominance frontiers from peers in adjacent semantic slabs (axis 1) while the Fibonacci axis ensures the crawl load is spread across all participating nodes in the source slab.

4.5 Token-economy tracking (SP_LATTICE_TOKENS)

With this gate on, the node tracks two separate ledgers of credit. The first, the **work-token ledger**, accumulates as the node performs verifiable inference on behalf of others — every CRT-shard contribution, every recall against a remotely-bound ARM slab, every block-attestation it serves to peers. The work ledger increments by an amount proportional to the verified compute performed, where the unit of verified compute is a normalised matmul-op count divided by a hardware factor that prevents trivial inflation from running on a faster machine. The second, the **discovery-token ledger**, accumulates when the node contributes a dominance-incomparable KSTE tree to the network — that is, when its local sieve identifies a cache cell that is novel against the global frontier. Discovery tokens reward bringing new information into the lattice rather than simply re-serving what is already known; they are the mechanism by which the network’s coverage of the input distribution grows beyond what any single operator could supply.

Both ledgers are local until block boundaries. At the periodic block cadence (see §6), the node submits a settlement proof: a Merkle-rooted summary of the deltas accrued since the last settlement, signed by the node key and accompanied by the dominance proofs (for discovery) or verification attestations (for work) that the chain validators will check. Accepted settlements mint the corresponding tokens against the node’s on-chain balance. The gate is off by default not because tracking is expensive — the bookkeeping is a few hundred bytes per layer per token — but because participation in the chain economy is opt-in: a node operator running the engine purely for local inference has no reason to incur the gossip and settlement costs. Turning the gate on is the operator’s explicit consent to be a paid participant.

Section 5: Network and protocol

The Lattice network is the carrier for everything in §4 once it crosses the boundary between one node and another. This section describes how the wire works.

5.1 DHT over prime-factored lattice key space

The DHT layer is described mathematically in §4.4; here we cover the operational shape. The Lattice runs a libp2p-compatible swarm: each node holds a long-lived Ed25519 identity key, derives a peer ID from it, and joins the DHT by contacting one or more configured bootstrap

peers. The transport is TCP with optional Noise encryption (default on for inter-node traffic, off for in-LAN benches where latency dominates and the messages are signed at the protocol layer anyway). QUIC is on the roadmap but not yet in production; the gain on inter-node latency is real (one round-trip saved per connection) but the maturity of the QUIC implementation on Windows hosts has not been validated.

The 2-axis routing of §4.4 is layered on top of standard Kademlia: the k-bucket table is partitioned by semantic axis (so each bucket holds peers whose semantic-slab prefix matches at the bucket's depth), and within each bucket the entries are sorted by Fibonacci-hash distance. Lookups walk the semantic axis first (closer prefix = closer logical distance) and break ties on the load axis. This composes cleanly with the standard Kademlia bootstrap and refresh logic; the only modification is the distance metric.

Peers are health-checked every 30 seconds with a small ping (16 bytes); a peer that misses three consecutive pings drops out of the routing table. Re-discovery is automatic via the next refresh cycle. The protocol is intentionally chatty rather than minimal because the underlying network conditions (residential broadband, intermittent NAT) are unreliable enough that aggressive health-checking pays for itself in fewer wedged sessions.

5.2 Wire formats

Three message types dominate the wire: KSTE trees, ARM updates, and CRT residue blobs.

KSTE trees are packed at 64 bytes per node: a 1-byte type tag, a 1-byte depth marker, a 30-bit fingerprint (split across two fp16-shaped fields for alignment), a 16-byte parent pointer (Merkle hash), and a 38-byte payload — either an internal-node delta blob or a Spinor leaf. The tree as a whole is delivered as a length-prefixed sequence of nodes in pre-order traversal, with the root first. A typical tree is 6-10 nodes deep, so the median tree message is roughly 500 bytes. The encoding is fixed-endianness (little-endian, the dominant target architecture) and there are no embedded pointers — everything is index-relative within the message.

ARM updates are a single ring element of R_q at the configured NTT N . For our standard $N=256$ and dual-prime q at 60-bit reconstructed, that is $256 \text{ coefficients} \times 8 \text{ bytes} = 2 \text{ KB}$ per slab delta. We do not currently delta-encode (the ring sum is the delta), though there is room to: subtracting the last-acknowledged remote slab from the local slab and shipping the difference would compress well-correlated traffic. This is one of the future optimizations.

CRT residue blobs are the per-layer activations during sharded inference. They are sized by the model's hidden dimension times 30 bits per scalar (packed); for Llama 3.1 8B ($d=4096$) that is $4096 \times 30 / 8 = 15 \text{ KB}$ per layer per token. At 32 layers and 100 tokens/sec target generation rate, that is 48 MB/sec of inter-shard bandwidth — comfortably within a gigabit LAN's capacity but well above what residential broadband can sustain. This bandwidth shape is why CRT sharding is LAN-bound in the current generation.

5.3 Gossip and propagation

For payloads other than CRT (which is point-to-point between sharded peers), the protocol is gossip. Each node maintains a small set of *fanout peers* — typically 8 randomly chosen from the routing table, refreshed every minute — and when a new payload arrives that the node has not seen before, it forwards the payload to its fanout peers (minus the one it came from). Deduplication is by content hash; each node maintains a recently-seen set of roughly 10,000 hashes with a TTL of 5 minutes, evicting LRU.

The 2-axis routing matters here too: gossip messages are tagged with their semantic slab, and fanout peers are selected preferentially from peers whose semantic slab overlaps the message's. This biases propagation toward peers who are likely to care, without precluding

rare cross-slab discovery (the 8-peer fanout is large enough to almost always include at least one out-of-slab peer per hop).

Block messages are special: they propagate aggressively (fanout of 32) and validate-on-receive (the receiving node checks the block's signatures, the validator selection, and a sample of work-token proofs before forwarding). A node that receives an invalid block does not forward it; this contains the damage from a misbehaving peer to its immediate neighborhood, where it will be observed by the slashing infrastructure described in §6.

Section 6: Blockchain design

This section is scaffolding. The chain exists to give federated participation an accounting layer; the parameters here are starting points, expected to be revised as the network grows. We have chosen to write down a specific design rather than gesture at one, because specificity is what makes the design critiquable. Where the choice is contested or unclear, we say so.

6.1 Two-token economy

There are two native tokens on the Lattice chain. The **Work token (W)** is paid out for verifiable inference contributions: serving a CRT shard, attesting blocks, responding to a remote ARM recall. The **Discovery token (D)** is paid out for contributing a dominance-incomparable KSTE tree to the global sieve — that is, for genuinely novel cache contributions that expand the network's coverage.

The two tokens are fungible against each other through an automated market maker (a Uniswap-V2-style constant-product pool), with both sides bootstrapped at genesis (see §6.5). The conversion ratio drifts with supply and demand; the design intent is that work and discovery should be roughly equally weighted at steady state, but the market is allowed to find the actual ratio.

The reason for two tokens rather than one is twofold. First, separating them lets validators tune emission policy: if the network is over-served (lots of work, few queries) then work emission can be throttled while discovery emission stays high to encourage cache expansion. Second, the two activities have different economic profiles — work is high-frequency, low-value-per-event; discovery is low-frequency, high-value-per-event — and conflating them in a single ledger would force every participant to optimize for the marginal of both, when in practice operators specialize.

A note on monetary policy: emission for both tokens follows a logarithmic schedule (block-reward shrinks slowly over time, asymptoting toward zero) rather than the abrupt halvings of Bitcoin. This is deliberate: we want predictable long-run dilution rather than the speculative volatility around halving events. The schedule is parameterised at genesis and updatable by validator vote with a six-month cooldown.

6.2 Proof-of-Useful-Work via dominance verification

The chain's core consensus primitive is not arbitrary hashing — it is *dominance verification*. To mint a Discovery token, a node must submit a KSTE tree along with a proof that the tree is incomparable against the chain's current dominance frontier (the on-chain Merkle-rooted summary of all accepted KSTE trees to date). The proof is small: a Merkle inclusion path from the chain's frontier root to the trees that bound the new tree's incomparability claim, plus the new tree itself. A validator checks the proof by recomputing the dominance comparisons against the cited bounding trees; if the new tree is genuinely incomparable, the proof verifies and the token is minted.

To mint a Work token, the node submits an *attestation* of work performed: a signed record of a CRT-shard session or an ARM-recall service, countersigned by the requester. The validator checks both signatures and ensures the work record’s hash is consistent with the requester’s claim. Work attestations are heavier than discovery proofs in volume but lighter per-attestation in compute; the chain budgets accordingly.

The key property is that the work being proved is *useful*: it is the same work that the user wanted done anyway. Compare to Bitcoin, where the proof-of-work is intrinsically wasteful (the SHA-256 hashing has no purpose other than rate-limiting block production). Lattice’s proof-of-useful-work means that the energy spent producing tokens is the energy spent producing the inference and discovery that is the network’s actual product.

The honest caveat: useful-work proof systems are harder to make adversary-resistant than wasted-work systems, because the adversary can compute on hardware that produces real outputs and is hard to distinguish from honest participation. The discovery side is partly protected by the dominance check (a forged “novel” tree must actually be incomparable to be accepted, and forging incomparability is no easier than discovering it), but the work side relies on requester countersignatures, which means a colluding requester-server pair can mint work tokens against fake sessions. The mitigation is rate-limiting and reputation, both described in §6.6.

6.3 Block structure

Blocks are produced at a target rate of one per 60 seconds. The variance of inter-block intervals under the Golden-Ratio validator rotation (§6.4) is bounded by the rotation’s discrepancy property, which is tighter than the Poisson process produced by a randomised beacon — useful because it means downstream consumers can rely on roughly-on-time blocks.

Each block contains:

- **Header.** Parent block hash, block number, timestamp, validator ID, validator signature, Merkle root of body.
- **Work attestations.** A sequence of work attestations (requester-server signed pairs) with their tokens minted accordingly.
- **Discovery commitments.** A sequence of KSTE-tree dominance proofs, each minting Discovery tokens to its submitter. The accepted trees become part of the next block’s dominance frontier root.
- **ARM gossip updates.** A summary of ARM slab updates seen on the network in the last block-interval. These are not consensus-critical (ARM is best-effort gossip), but they are written into the block as a checkpoint so a fresh-joining node can reconstruct the network’s current associative memory state without re-receiving the entire gossip history.
- **Sieve cache deltas.** Similarly, a summary of new KSTE trees accepted into the global sieve since the last block. These determine the next block’s dominance frontier root.
- **Token mint events.** The settlement records that turn the work and discovery accruals into on-chain balance changes.
- **Validator vote slot.** Reserved space for governance votes (parameter updates, validator-set changes).

A typical block is on the order of 100 KB to 1 MB depending on network activity. Block storage grows linearly with time but the dominance frontier — the load-bearing state for new-block validation — is bounded by the natural growth rate of incomparable trees, which empirically saturates as coverage grows. We have not seen full saturation in the bench; the projection is that the frontier reaches a stable size in the low millions of trees and stops growing meaningfully thereafter.

6.4 Consensus and validator rotation

Consensus is BFT-style: a quorum of two-thirds of stake-weighted validators must sign a block for it to be canonical. The validator for any given block — the proposer — is chosen by Golden-Ratio rotation through the validator set, described next.

6.x Validator selection via Golden-Ratio rotation

Stake-weighted validator selection in standard consensus protocols requires a verifiable random beacon (VRF, randao, or similar) — a non-trivial source of consensus compute. By Three-Gap (PPT-LAT-Theory T7), we can replace the random beacon with deterministic Golden-Ratio rotation through the validator set: with stake-weighting baked into the interval lengths, stepping through the set by increments of φ guarantees mathematically optimal fairness and distribution over time without relying on pseudo-randomness.

Concretely: let V be the validator set with stake weights $w_1, \dots, w_n$ normalised so $\sum w_i = 1$. Partition the unit interval into n arcs of length w_i . To select the validator for block b , compute $r_b = \{b \cdot \varphi\}$ (fractional part) and pick the validator whose arc contains r_b . Three-Gap guarantees this produces a uniform distribution of validator selections over time, weighted exactly by stake, with no manipulability advantage from controlling any single block.

The advantage over a random beacon is twofold: (1) no consensus rounds needed to agree on the beacon output for the next block, and (2) any node can independently verify the validator assignment for any block by computing $\{b \cdot \varphi\}$ — no commit-reveal, no slashing for beacon non-cooperation.

The validator set itself is updated by stake delegation: token holders bond W or D against a validator's address, and the validator's stake weight w_i is the sum of bonds. Bond and unbond are processed at block boundaries with a configurable unbonding delay (default 14 days) that prevents stake from being moved fast enough to manipulate near-term validator selection.

6.5 Genesis and parameterization

Genesis parameters are chosen to bootstrap a small functioning network rather than to optimize for any particular long-run equilibrium; the expectation is that everything in this section will be revised as the chain grows.

Token supply at genesis. 100,000,000 W and 100,000,000 D, distributed across a small founder set, an early-contributor set, and a treasury (held by the validator quorum for protocol grants). The founder share is restricted by linear vesting over 4 years.

Initial validator set. 21 validators, geographically distributed where possible, with stake bootstrapped from the treasury. The set expands by validator-vote as the network grows; the design ceiling is in the low hundreds of validators, above which BFT signature aggregation becomes the binding bottleneck.

NTT and CRT parameters. $N = 256$, $q_1 = 1073738753$, $q_2 = 1073732609$, reconstructed modulus $\approx 2^{60}$. These match the engine's standing configuration and the chain's residue-verification path uses the same arithmetic, so a validator can re-execute any submitted work-attestation on a single machine if it wants to.

KSTE configuration. Tree depth 8, fingerprint width 30 bits, Spinor block 63 bytes. These match the production cache encoder; the chain's dominance verification is therefore the same operation the engine already performs on every cache write.

Block parameters. 60-second target inter-block time, 2 MB block size cap, two-thirds-stake BFT quorum, 14-day unbonding delay.

These constants are encoded in the genesis block and can be modified by a validator vote with a six-month cooldown, except for the NTT/CRT primes which are fixed for the lifetime of the chain (changing them would invalidate every prior dominance proof).

6.6 Slashing

Validators and participants can lose stake for misbehavior. The chain defines four slashing conditions:

False novelty claims. A node submits a KSTE tree claiming dominance-incomparability against bounding trees that, when re-checked, do not bound the claim correctly. The submitter loses a fraction of bonded stake (default 10%) and the false claim is purged from the frontier.

Residue forgery. A CRT-shard server submits residue blobs that fail the cross-prime consistency check at the requester end. The server loses a larger fraction of stake (default 50%) and is removed from the validator set if the offense recurs.

Equivocation. A validator signs two different blocks at the same block height. The validator loses its entire stake; this is the most serious offense because it threatens consensus directly.

Censorship. A validator that is the rotation-elected proposer for a block but produces no block within the block-interval is mildly penalised (default 1% of stake) and the next-elected validator (the next Golden-Ratio arc) proposes instead. Repeated censorship leads to validator removal.

Slashing decisions are made by the validator quorum based on evidence submitted by any node; a successful slash mints a small bounty to the evidence submitter, paid from the slashed stake. This is the chain's whistleblower incentive.

6.7 Mutability — papers are scaffolding, not specification

We have emphasized throughout that this design is scaffolding. The slashing fractions, the block size cap, the unbonding delay, the emission schedule — all of these are starting points. We expect that within the first six months of mainnet operation, real adversarial pressure will reveal which constants are tight and which are slack. The validator-vote mechanism is the path for revising them.

What is *not* mutable: the NTT/CRT primes (changing them invalidates prior dominance proofs), the dominance comparison itself (it is the mathematical core), and the existence of two separate token ledgers (folding them together is a fundamental design change, not a parameter tweak). Anything else is in scope for revision.

This is what we mean by “papers are scaffolding, not specification.” The mathematical core (PPT-LAT-Theory) is fixed; the systems paper is current; the chain parameters are a snapshot of the design at this date and will be revised. A future systems paper will reflect those revisions rather than pretending the constants were oracular from the start.

Section 7: Failure modes and mitigations

Every networked system has failure modes; a useful systems paper names them rather than pretending they don't exist.

Sybil attacks. An adversary spawns many cheap identities to dominate the DHT and the validator set. Mitigation: stake-weighting on the validator side means Sybil identities have no consensus weight unless they buy stake on open market; on the DHT side, the 2-axis routing of §4.4 limits the damage a single semantic-slab cluster can do, and the Fibonacci-load

axis spreads any honest traffic across the cluster's nodes rather than concentrating it on the adversary's. Sybils can still wedge gossip — by accepting and dropping payloads instead of forwarding — but the bound on damage is local rather than global.

Cache poisoning. An adversary submits KSTE trees designed to be incomparable but to point toward arbitrary cache content, polluting the global sieve. Mitigation: dominance verification at write time means the cell content matters, not just the tree shape; a poisoned cell still has to be incomparable on its actual content, which limits the adversary to genuinely novel-by-content submissions. This doesn't prevent semantic poisoning (a cell that is genuine-but-misleading), but it does prevent the most trivial attacks.

Eclipse attacks. An adversary surrounds a target node with adversarial peers, controlling its view of the network. Mitigation: the 2-axis routing makes eclipse harder than in single-axis Kademlia, because the target's routing table is partitioned by semantic slab and the adversary must control sufficient peers in each slab to be effective. Additionally, the bootstrap-peer flag accepts a comma-separated list of peers, and the target node prefers diverse bootstrap peers (different ASNs, different geographic origins where available) for the initial routing-table seed.

Free riders. Operators run nodes that consume bandwidth and serve queries but never propagate gossip or contribute KSTE trees back to the network. Mitigation: this is a recurring problem in P2P systems and the only known robust solution is incentive alignment, which is what the two-token economy provides. A node that does not gossip earns no work tokens; a node that does not discover earns no discovery tokens. Pure consumers are tolerated (the network can absorb them at the margin) but they cannot scale, because they have no on-chain identity to bond stake against.

ARM capacity overflow. As more bindings are gossiped into a node's ARM bank, recall noise grows. Mitigation: periodic rebinding (§4.2) bounds the working set; nodes that wish to retain long-term associative memory must do so out-of-band (a separate persistence layer outside the gossiped slab). The chain itself does not store ARM state, only summary checkpoints in blocks.

Cache staleness. A KSTE tree that was incomparable when submitted may become dominated as the frontier grows. The chain treats this as natural state change — the original Discovery token is not clawed back, but the tree's contribution to the current frontier is just whatever it currently is. This is the right behavior because it preserves the property that Discovery rewards genuine novelty at the time of contribution.

Network partitions. A split in the gossip layer (caused by a major ISP outage or backbone failure) causes the network to operate as two disjoint sub-networks for some interval. Mitigation: the chain uses a BFT quorum, so a sub-network with less than 2/3 stake cannot produce blocks. When the partition heals, the longer-chain rule applies (the sub-network that did produce blocks wins). Sub-network participants who minted tokens during the partition are not penalized for the partition itself, but their gossip-deltas are only applied to the canonical chain after the partition heals.

Collusion. A coalition of validators with combined stake greater than 1/3 can stop block production by refusing to sign; a coalition with combined stake greater than 2/3 can produce arbitrary blocks. Mitigation: this is the standard BFT bound and we don't claim to do better than it. The chain's defense is the social one of decentralized stake distribution; the chain protocol provides no cryptographic defense against a stake-supermajority cartel. The treasury holdings at genesis are deliberately distributed to make supermajority capture expensive.

Section 8: Comparison to prior work

The Lattice sits in a crowded design space. This section names the closest neighbors and where the Lattice diverges.

Hivemind / Petals. These projects pioneered the idea of running large LLMs across volunteer GPUs over the public internet. Petals in particular ships working multi-node inference across heterogeneous home GPUs for BLOOM-176B and Llama-class models. The Lattice differs principally in (a) the cache and weight compression that lets it run on phone-class hardware, not just GPU peers, and (b) the on-chain accounting layer that allows participants to be paid for contributions, which Petals deliberately avoided. Petals is a closer-to-academia project; the Lattice is a closer-to-production one. We have used Petals as a reference for the gossip and routing layer design, but the wire formats and consensus model are not shared.

Bittensor. Bittensor implements a peer-to-peer market for ML model outputs with on-chain incentives. The closest analogue to our work-token. Bittensor's contribution is the idea that contribution can be measured by peer-ranking against a shared benchmark; the Lattice's discovery-token has the same flavor but measures contribution by dominance frontier expansion rather than by peer ranking. The two systems could coexist — a Bittensor subnet running on top of a Lattice cache substrate is plausible — but they are not the same system.

DiLoCo. Federated training rather than inference, but the geographically-distributed compute story is similar. DiLoCo's contribution is showing that the gradient-sync frequency can be relaxed without hurting convergence, which lets training tolerate WAN latencies. The Lattice's CRT-sharded inference faces an analogous latency-tolerance question, and we are watching DiLoCo's results to inform our roadmap on relaxed-synchronization sharding. DiLoCo is research; the Lattice is a production target with a research roadmap.

Filecoin / Storj. Storage networks rather than compute, but the on-chain accounting layer for verifiable-useful-work is the closest precedent for what we are doing with proof-of-useful-work. Filecoin's proof-of-replication and proof-of-spacetime are different cryptographic primitives but the same systemic role — they are the chain's way of verifying that the network's product (stored data, in Filecoin's case) was actually produced. We owe Filecoin a conceptual debt for showing that this design pattern is viable at production scale.

YaCy. A peer-to-peer search engine, dormant but instructive. YaCy showed that a distributed crawl and index can be operated by a volunteer network; it failed to reach critical mass partly because there was no incentive layer to reward crawlers. The Lattice's discovery-token is in part a response to that lesson: making the cache contribution itself the rewarded act, rather than relying on volunteer altruism.

Render Network. A blockchain-coordinated marketplace for GPU rendering work. Closer to a generic compute marketplace than to Lattice's tightly-coupled inference fabric, but it is the closest production example of compute-for-tokens at scale. Render's accounting layer is simpler than ours (no useful-work proof; the work is just verifiably-rendered frames) and Render's compute jobs are embarrassingly parallel in ways inference is not. The mechanisms are different but the political economy is similar.

Bitcoin / Ethereum. The base layer. Bitcoin's contribution is the validator-rotation primitive; Ethereum's contribution is the smart-contract substrate that lets economic logic be expressed declaratively. The Lattice borrows the BFT-quorum consensus model from Ethereum's post-merge design, the on-chain settlement-record pattern from both, and the validator-slashing playbook from both. The Lattice does not currently have a general-purpose smart-contract layer; the chain logic is fixed at protocol level. Adding a general VM is on the longer-term roadmap but is not a near-term priority because the use case is small (governance votes and treasury management, mainly).

Section 9: Open questions

This section names what we don't know. We name them rather than gesture at them because being precise about what is unknown makes the work bounded.

Constants under real churn. The slashing fractions, unbonding delay, fanout sizes, and gossip refresh intervals are all educated guesses at this point. We do not have data from a churning production network to validate any of them. We will know more after six months of mainnet operation; until then, these constants are placeholders.

Two-token equilibrium proof. We have argued informally that a two-token system with separate work and discovery rewards should not collapse to a single token via the AMM, because the two activities have different supply dynamics. We do not have a formal equilibrium proof, and it is possible that under some demand regime the discovery side dries up (if all easy novel content has been discovered) or the work side does (if there is no demand for inference). The chain has parameter levers to respond to this, but we have not characterized when those levers need to be pulled.

Slab assignment under adversarial churn. The 2-axis DHT assumes that semantic slabs are stable enough that a node's slab assignment is a meaningful long-term identity. If churn is very high — nodes leaving and rejoining with different IDs frequently — the slab assignment becomes noisier and the routing benefit degrades. We do not know what level of churn breaks this; the literature on Kademlia variants under churn is helpful but doesn't directly address the 2-axis case.

Real-time vs batch tradeoff for CRT sharding. §4.3 frames CRT sharding as LAN-bound for interactive use. Real-time WAN sharding would require relaxing per-layer synchronization, which is an open numerical-accuracy question. We have not characterized how much synchronization can be relaxed before perplexity drift becomes unacceptable.

Validator hardware diversity. Block validation requires re-checking dominance proofs and re-executing CRT residue checks. If validators all run the same hardware, a hardware bug affects every validator simultaneously, which is a consensus risk. We want validators to run a diversity of hardware (Intel/AMD/ARM, different vendors of compute accelerators) to provide implementation diversity at the consensus layer. We do not know how to incentivize this beyond exhortation; making it economically rational is an open design question.

Privacy of shared KV cache. A shared cache substrate inherently leaks information about what was queried: an adversary monitoring the gossip layer learns the semantic distribution of queries on the network. For consumer use cases this is acceptable (the queries are not private), but for enterprise or regulated workloads it is not. We do not currently have a privacy-preserving variant; the most plausible route is to la

Appendix A — L1 ABI contract (PPT-LAT-L1-ABI v0)

The Layer-1 (math core) C ABI contract, frozen for the duration of Phase 2. This is the boundary between libshannonprime (C/CUDA/ Vulkan/HVX) and any L2 binding (primary target: Rust). The full standalone document lives at papers/PPT-LAT-L1-ABI-v0.md; the body is reproduced verbatim here so the Systems paper carries the locked contract without an external reference.

The Layer-1 (math core) C ABI contract for Shannon-Prime. Locks the boundary between libshannonprime (C/CUDA/Vulkan/HVX) and any L2 binding (primary: Rust). Designed against the three tear-down axes: caller-allocates memory ownership, Send-but-not-Sync session handle, and an error surface that names VHT2 / Spinor / Frobenius / ARM / sieve failure modes explicitly. After this contract is signed off, .sp-model byte layout falls out mechanically.

1. Object lifetimes

Two opaque types cross the FFI:

```
typedef struct sp_model    sp_model;    // read-only after load; many sessions per model
typedef struct sp_session sp_session;  // single-thread state: KV + ARM + sieve + arch s
```

Construction is L1's responsibility (only L1 knows the internal layout). Destruction is the caller's responsibility but always via the matched destroyer; the caller never free()s a returned pointer directly.

```
sp_status sp_model_load      (const char* sp_model_path,
                             const char* sp_tokenizer_path,
                             /*out*/ sp_model** out);
void      sp_model_unload    (sp_model*);

sp_status sp_session_create (const sp_model*,
                             const sp_session_config*,
                             volatile _Atomic bool* cancel_flag, // L2-owned, see §5
                             /*out*/ sp_session** out_session);
void      sp_session_destroy(sp_session*);
```

Cancel-flag lifetime contract. The cancel_flag pointer must remain valid (and the memory backing it must not be freed) for the entire lifetime of the sp_session — i.e. until sp_session_destroy returns. L2 owns the atomic; L1 only reads it. There is no cancel handle that L1 allocates, so there is no UAF window between a session being destroyed on one thread and a stale cancel handle being used on another. The L2 wrapper holds the atomic inside an Arc (see §5 and §8) which guarantees the address stays live for as long as either the session or any cancel-clone is alive.

2. Memory ownership — caller-allocates everywhere on the hot path

Every buffer crossing the FFI on the per-step path is caller-allocated. L1 never mallocs anything L2 has to free. Sizing comes from sp_model_arch before session creation:

```
sp_status sp_model_arch (const sp_model*, /*out*/ sp_arch_info*);
```

sp_arch_info carries vocab_size, hidden_dim, n_layers, n_heads, n_kv_heads, head_dim, rope_base_microcents, swa_window, ffn_variant, norm_variant, tied_embeddings, and the arch enum. L2 reads it once at load, sizes the logits buffer as vocab_size * sizeof(float), and never re-queries.

The out parameter is caller-stack-allocated by C convention; L1 populates the struct in place. Rust bindings should declare this as &mut MaybeUninit<sp_arch_info> rather than &mut sp_arch_info so the borrow checker doesn't assume the struct is initialized on entry to the call (it isn't — L1 fills it). After sp_status == SP_OK, the binding may call MaybeUninit::assume_init.

The only L1-allocated memory crossing the boundary is the three opaque handles above, each with a matched destroyer. There is no sp_alloc / sp_free pair in the public ABI — internal arenas (activation, ARM bank, sieve, Spinor pool) are session-private and released by sp_session_destroy.

3. Forward pass — two-function ABI

```
sp_status sp_prefill_chunk (sp_session*,
                           const int32_t* tokens, size_t n_tokens,
                           /*out, caller-allocated*/ float* logits_last,
                           size_t logits_capacity);

sp_status sp_decode_step   (sp_session*,
                           int32_t token,
                           /*out, caller-allocated*/ float* logits,
                           size_t logits_capacity);
```

`sp_prefill_chunk` consumes `n_tokens` tokens, advances internal position by `n_tokens`, writes the last token's logits only. `sp_decode_step` consumes one token, advances by one, writes that token's logits. Two functions because their cost shape is asymmetric (compute-bound vs bandwidth-bound) and L2 must be free to interleave them across requests. `logits_capacity < vocab_size` returns `SP_EBADARG`.

4. Session manipulation — speculative-decoding-shaped from day one

```
sp_status sp_session_clone   (const sp_session*,
                              volatile _Atomic bool* cancel_flag,    // L2-owned, see §5
                              /*out*/ sp_session** out);
sp_status sp_session_rewind  (sp_session*, size_t n_tokens);
sp_status sp_session_position (const sp_session*, /*out*/ size_t* pos);
```

`sp_session_clone` is the spec-decode fork primitive: deep-copy KV + ARM + sieve, return an independent session. `sp_session_rewind` is the spec-decode reject primitive: roll back `n_tokens` of accepted state. ARM writes are journaled per-token so rewind is precise, not “drop the whole bank.” Getting these in v0 is what makes spec-decode tractable.

5. Cancellation — L2-owned atomic flag, no L1 call

There is no `sp_session_cancel()` function. The cancel surface is the `volatile _Atomic bool* cancel_flag` pointer L2 passes into `sp_session_create` (§1). L2 owns the storage; L1 only reads it.

```
// L2 side – sketch
let flag: Arc<AtomicBool> = Arc::new(AtomicBool::new(false));
let flag_ptr: *const AtomicBool = &*flag;                // stable for the Arc's lifetime
let sess = sp_session_create(model, &cfg, flag_ptr.cast(), &mut out_sess);

// HTTP-handler thread holds a clone of the Arc and cancels by:
flag_clone.store(true, Ordering::SeqCst);                // pure Rust, no FFI call
```

L1 reads the flag at every layer boundary in `sp_prefill_chunk` and every step in `sp_decode_step`. On observing true, the in-flight L1 call unwinds to the last completed boundary and returns `SP_ECANCEL`. Session state at that point is consistent (last layer fully applied), not partial.

Why this shape is UAF-proof. A naive design with an L1-allocated `sp_cancel*` paired to a session creates a window:

1. Worker thread drops the Session, `sp_session_destroy` runs, frees the C-side cancel handle.

2. Handler thread, milliseconds later, calls `sp_session_cancel(stale_ptr)` — dangling pointer dereference.

Inverting ownership eliminates the window. The atomic lives inside an `Arc<AtomicBool>`; the heap allocation backing an `Arc` never moves and is only freed when the *last* clone drops. Both the Session wrapper and the Cancel wrapper hold a clone, so the address L1 reads from remains valid until both are gone. Setting the flag on a already-destroyed-session `Arc` is a harmless write to memory nobody is reading anymore.

The cancel-flag read pattern in L1 is `__atomic_load_n(flag, __ATOMIC_RELAXED)` at boundaries (or `_InterlockedOr` on MSVC) — no fence required; the boundary itself is the synchronization point.

6. Determinism

Set at session create, immutable for the session’s lifetime:

```
typedef struct {
    size_t    max_context;
    bool      deterministic;           // serial reductions, single stream, no atomic-add
    uint32_t  arm_bank_kb;            // 0 = arch default
    uint32_t  sieve_capacity;         // 0 = arch default
    uint32_t  flags;
} sp_session_config;
```

`deterministic=true` is what `T_FRO_4` runs against (bit-exact gate). Production runs with `deterministic=false` (ULP-tolerance gate). Toggling mid-session is forbidden because reduction order and stream topology are baked into kernel selection at create time.

7. Error surface

`sp_status` is a signed int. `SP_OK = 0`. Negative = error. Positive reserved for future “soft” signals (e.g. “ARM bank approaching capacity, consider a write_stride bump”). All failing calls also set a thread-local error string retrievable via:

```
const char* sp_last_error(void);    // pointer valid until next L1 call on this thread
```

Enum (covers the VHT2 / Spinor decompression surface explicitly):

```
typedef enum {
    SP_OK                      = 0,

    // Generic
    SP_ENOMEM                  = -1,
    SP_ECANCEL                  = -2,
    SP_EBADARG                  = -3,
    SP_EBADSTATE                = -4,
    SP_EUNSUPPORTED            = -5,
    SP_EIO                      = -6,

    // Model load / arch
    SP_EBADFORMAT               = -10,    // sp-model magic/version mismatch
    SP_EBADARCH                 = -11,    // arch_id not recognized
    SP_ETOKENIZER_HASH          = -12,    // sp-tokenizer sha256 ≠ sp-model.tokenizer_hash
    SP_EVOCAB                   = -13,    // tokenizer vocab size ≠ model vocab size
}
```

```

// Discrete algebra layer – the “we lost the algebraic invariant” surface
SP_ESPINOR_BADBLOCK = -20, // 63-byte Spinor block parity/CRC mismatch
SP_EVHT2_DOMAIN     = -21, // VHT2 inverse out-of-range
SP_EMOBIUS_PERM     = -22, // Möbius reorder index invalid
SP_EOK_NORM         = -23, // 0_K element norm overflow
SP_EFROBENIUS_QUANT = -24, // Frobenius dequant scale/shift invalid
SP_ENTT_OVERFLOW    = -25, // CRT NTT residue overflow (defensive – should be impo
SP_ERING_DEGREE     = -26, // R_q polynomial degree mismatch

// Lattice / framework features
SP_ESIEVE_FULL      = -30, // Friedman sieve full + eviction policy refused
SP_EARM_BANK_FULL   = -31, // ARM HRR bank exhausted
SP_EDOMINANCE_CYCLE = -32, // □_d encountered a non-wqo input (corrupt KSTE)
SP_ECONTEXT_FULL    = -33, // sequence position == max_context; L2 should trigger
                        // Fibonacci sub-sampling eviction and retry

// Backend
SP_ECUDA            = -40, // wraps any cudaError_t; sp_last_error has the detail
SP_EVULKAN          = -41,
SP_EHVX             = -42,
SP_EBACKEND_OOM     = -43, // device-side OOM, distinct from host SP_ENOMEM
} sp_status;

```

The discrete-algebra block (−20..−26) is the one that matters for correctness verification. Every gate in PPT-LAT-Theory T1..T7 / E9.x / E10 maps to one of those return codes if it trips at runtime.

SP_ECONTEXT_FULL is structurally distinct from SP_ENOMEM: the former is “the sequence position counter hit max_context”, and L2’s correct response is to trigger Fibonacci sub-sampling eviction (per Roadmap §20.x golden-ratio KV retention) on the session’s KV + ARM arenas and reissue the prefill/decode call. The latter is “the host allocator returned NULL” and is a hard fatal. Collapsing them would make the eviction-on-context-full policy unimplementable from L2.

8. Threading model — what L2’s Rust wrapper looks like

```

use std::sync::Arc;
use std::sync::atomic::{AtomicBool, Ordering};

pub struct Model { ptr: *const sp_model }

pub struct Session {
    ptr: *mut sp_session,
    cancel: Arc<AtomicBool>, // held so the storage L1 reads stays live
}

#[derive(Clone)]
pub struct Cancel(Arc<AtomicBool>); // a clone of Session.cancel

impl Cancel {
    pub fn cancel(&self) { self.0.store(true, Ordering::SeqCst); }
}

```

```
impl Session {
    pub fn cancel_handle(&self) -> Cancel { Cancel(self.cancel.clone()) }
}

unsafe impl Send for Model {} unsafe impl Sync for Model {}
unsafe impl Send for Session {} // not Sync
// Cancel is Send + Sync automatically (Arc<AtomicBool> is Send + Sync)
```

Model is Send + Sync because it is immutable after `sp_model_load` returns. Session is Send (workers can hand it between threads between calls) but **not** Sync (no two threads may be inside `sp_prefill_chunk` / `sp_decode_step` on the same session simultaneously — enforced by `&mut self` on every step method). Cancel derives Send + Sync from its `Arc<AtomicBool>` field — no unsafe impls required, and no FFI call inside `Cancel::cancel`.

The HTTP handler thread holds a Cancel. The inference worker thread holds the Session. Both internally point at the same `Arc<AtomicBool>` storage, which L1 reads via the raw pointer it was given at `sp_session_create`. The Arc's allocation is stable for as long as either side holds a clone, so the L1-side pointer never dangles. Cancellation from L3 is a single relaxed atomic store; no lock, no context switch, no FFI crossing.

9. What this contract does *not* commit to

Out of scope for v0, deliberate:

- **Streaming logits** — L2 may want token-by-token streaming callbacks. For v0 the caller polls. A callback variant (`sp_decode_step_cb`) can be added without breaking this ABI.
- **Multi-session batching at L1** — v0 is one-session-per-call. Cross-session prefill batching (vLLM-style continuous batching) is a v1 concern and goes in a new function, not a modification of `sp_prefill_chunk`.
- **In-process sampler** — L1 emits logits and stops. Any sampler (temperature, top-k, top-p, mirostat, grammar-constrained) is L2.
- **Tokenization** — L2 owns the SentencePiece / BPE blob via the `.sp-tokenizer` sibling file. L1 only verifies the hash.
- **Telemetry** — no `sp_metrics()` in v0. L2 instruments around the FFI boundary.

10. Sign-off checklist

Before this is locked, three reads against the tear-down axes:

1. **Memory ownership.** Grep this doc for any out parameter that isn't either an opaque-handle constructor (`model/session/cancel`) or a caller-allocated buffer. There should be zero. If you find one, name it.
2. **Send/Sync compatibility.** Walk the four-line Rust block in §8. Every L1 function called from each thread must be safe given the marker traits asserted. Cancellation is the one that needs the hardest stare.
3. **Error surface vs failure modes.** Walk the eight load-bearing theorems (T1..T7 + E10) and confirm every runtime failure path from a theorem invariant has a named status code. Anything that would currently return `SP_EBADSTATE` and lose information is a gap.

Once those three are clean, `.sp-model` byte layout becomes:

```
magic          "SPMD"                // 4 bytes
version        u32                    // major.minor packed
arch_id        u32                    // sp_arch_info.arch_id
arch_struct    u8[256]                // memcpy'd into sp_arch_info
```

```

tokenizer_hash  u8[32]                // sha256 of paired .sp-tokenizer
tensor_count    u32
tensor_table    sp_tensor_entry[N]    // (offset, dtype_id, shape, name_hash)
tensor_data     [64-byte aligned; Spinor blocks pad 63→64]

```

— and is ~30 minutes of work.

Status. v0 draft. Not yet integrated into PPT-LAT-Systems §1 or the per-backend Phase-2 tracks. Sign-off goes against §10’s three checks; on green, this becomes Appendix A of PPT-LAT-Systems and the L1 ABI is frozen for the duration of Phase 2.

Appendix B — .sp-model byte layout (PPT-LAT-SP-MODEL v0)

The on-disk byte layout for .sp-model and the paired .sp-tokenizer, locked alongside Appendix A. Designed so sp_model_load is implementable as open + mmap + parse header + set up pointers with zero allocation proportional to tensor data size. The full standalone document lives at papers/PPT-LAT-SP-MODEL-v0.md; the body is reproduced verbatim below.

Companion document to PPT-LAT-L1-ABI-v0. Defines the on-disk byte layout for .sp-model and the paired .sp-tokenizer such that sp_model_load is implementable as open + mmap + parse header + set up tensor table pointers with zero allocation beyond the opaque sp_model handle itself. Every structural decision here is driven by the locked ABI (caller-allocates, mmap-friendly, no per-call file IO).

The Spinor 63→64 padding question — on-disk pad vs scatter-at-load — is decided in §6.

1. Goals and non-goals

Goals.

- sp_model_load is pure mmap + pointer setup. No malloc for tensor data. No memcpy of tensor data. The file IS the in-memory layout.
- Every byte the math kernels read at runtime is already at the correct alignment the moment mmap returns.
- Header parsing is a single memcpy into a fixed struct.
- Tensor lookup is $O(\log N)$ via name-hash binary search on a fixed 256-byte tensor table entry.
- The format is forward-compatible: v0 readers can refuse v1 files cleanly via the version field; v1 readers can fall back to v0 semantics by ignoring reserved bytes.
- Disk overhead is $\leq 2\%$ vs the raw packed tensor data. Spinor padding is the dominant contributor at ~1.5%.

Non-goals.

- Compression on disk. PPT weights are already compressed (Q8/Q4/Spinor); zstd-on-top buys little and breaks mmap.
- Encryption. Use OS-level (filesystem) encryption if needed.
- Multi-version-in-one-file. Each .sp-model is one model snapshot.
- Streaming load from network. Caller mmmaps a local file; remote-fetch is L2/L3’s concern.
- Tokenizer content. That’s .sp-tokenizer’s file; here we only carry its SHA-256.

2. Byte order, alignment, file extension

Endianness. All multi-byte integers and floats are little-endian. Target hardware (x86_64, ARM64, CUDA hosts, Hexagon V69) is uniformly little-endian in practice. Big-endian platforms must byte-swap at load or refuse the file. We do not pay portability cost for systems we will not ship to.

Alignment.

- **Header** lives at file offset 0, fixed 512 bytes (one sector).
- **Tensor table** is 64-byte-aligned, fixed at file offset 512.
- **Tensor data region** is 65536-byte-aligned (one Windows MapViewOfFile granularity unit). Each individual tensor inside it is 64-byte-aligned.

The 65536-byte data-region alignment matters specifically on Windows: partial mmmaps (MapViewOfFile) require the base offset to be a multiple of the allocation granularity (64 KB on Windows; 4 KB on Linux). Aligning the data region to the larger constant means a single MapViewOfFile call can map any tensor independently without header-region overhead.

File extension. .sp-model for the weights + arch + tokenizer hash, .sp-tokenizer for the paired tokenizer blob. Both files together constitute a deployable PPT model.

3. File header (fixed 512 bytes)

offset	size	field	notes
0	4	magic	ASCII "SPMD" (0x44 0x4D 0x50 0x53 LE)
4	2	version_major	u16, v0 = 0
6	2	version_minor	u16, v0 = 1
8	4	header_size	u32, total bytes of this header = 512
12	4	arch_id	u32, sp_arch_info.arch_id enum (LLAMA3=1, QWEN3=2, GEMMA3=3, DEEPSEEK_V4=4, ...)
16	4	arch_struct_size	u32, in bytes; for v0, sizeof(sp_arch_info)
20	4	arch_struct_capacity	u32, on-disk reservation = 256
24	256	arch_struct	memcpy-direct payload for sp_arch_info; unused tail zero-filled to 256
280	32	tokenizer_hash	u8[32], SHA-256 of paired .sp-tokenizer (entire .sp-tokenizer file, byte-by-byte)
312	4	vocab_size	u32, mirrors arch_struct field for fast pre-allocation; loader asserts equality
316	4	tensor_count	u32, number of entries in tensor table
320	8	tensor_table_offset	u64, byte offset of first tensor entry (= 512 in v0)
328	8	tensor_data_offset	u64, byte offset of first tensor byte (multiple of 65536)
336	8	file_size	u64, total file size in bytes; loader asserts == stat(file).st_size
344	8	created_unix_seconds	u64, wall-clock seconds since epoch at transcode time
352	8	transcoded_from	u64, hash of upstream file path (e.g. fxhash of the GGUF source path); zero if model was generated natively
360	4	header_crc32	u32, CRC-32 of bytes [0, 360) using standard IEEE polynomial; the field itself is excluded from coverage
364	148	reserved	zero-filled in v0

512 ... -- tensor table starts here

All offsets and sizes are absolute byte offsets from file start.

Why header_size is explicit and the reserved tail exists. v1 may extend the header by reusing reserved bytes. v0 readers memcpy(512 bytes) and ignore tail-extension fields they don't recognize. v1 readers compare header_size == 512 and run v0-compat parsing on match, v1 parsing on mismatch (which implies new fields after byte 512). This is forward-compat without a parser dispatch table.

Why CRC-32 of the header only. Per-tensor integrity is covered by the BLAKE3 hash in each tensor entry (§4). The header CRC catches gross corruption of the header itself (page tear, partial write during transcode) before the loader tries to interpret tensor offsets.

4. Tensor table (256-byte entries)

Tensor table entries are fixed-size so the table is a flat array addressable as entry[i] = tensor_table_offset + i * 256. No string table, no variable-length records, no second-level indirection.

```
typedef struct {
    char      name[80];           // bytes 0-79:  null-terminated tensor name
                                   // (longest GGUF names are ~30 chars; 80 is
    uint32_t  dtype_id;          // bytes 80-83:  enum, see §5
    uint32_t  n_dims;            // bytes 84-87:  rank, 1..8
    uint64_t  dims[8];           // bytes 88-151: shape in elements (not bytes);
                                   // unused entries are zero, NOT one
    uint64_t  offset_in_data;     // bytes 152-159: byte offset from tensor_data_offset;
                                   // multiple of 64
    uint64_t  size_bytes;        // bytes 160-167: on-disk byte length, including
                                   // any per-block padding (Spinor +1)
    uint32_t  block_size;        // bytes 168-171: on-disk granularity (64 for Spinor,
                                   // 32 for Q4 row-block, 1 for fp16/fp32)
    uint32_t  block_count;       // bytes 172-175: size_bytes / block_size, sanity
    uint8_t   blake3[32];        // bytes 176-207: BLAKE3-256 of the tensor's
                                   // on-disk bytes (size_bytes long,
                                   // starting at tensor_data_offset +
                                   // offset_in_data)
    uint64_t  name_hash;         // bytes 208-215: xxh3_64(name); table is sorted
                                   // by this for binary lookup
    uint8_t   reserved[40];      // bytes 216-255: zero-filled in v0
} sp_tensor_entry;
// sizeof = 256
```

Entries are sorted by name_hash ascending. Loader binary-searches by hash, then verifies name equality on the match (defends against the 1-in-2⁶⁴ hash collision). Sort + hash means lookup is O(log N + 1 strcmp), which on a 2000-tensor model is ~11 hash comparisons and one string compare.

Why name is fixed at 80 bytes. GGUF's longest tensor names (blk.NN.ffn_gate_inps.weight and variants) fit in 40 bytes; 80 gives headroom for compound names from MoE / SSM / hybrid architectures without going variable-length.

Why dims is u64. Allows >4G-element tensors without ambiguity. For a vocab projection on a 32B model, dims[0] * dims[1] already exceeds 2³².

Why per-tensor BLAKE3 rather than CRC. BLAKE3 is fast on modern hardware (~5

GB/s/core), cryptographically strong, and catches the specific failure modes we care about (bit-rot, partial writes, model provenance). Verifying every tensor on load is opt-in via `sp_session_config.flags & SP_VERIFY_TENSORS`. The default load trusts the file and just checks the header CRC.

5. dtype_id enum

```
typedef enum {
    // Continuous (parity with GGUF for direct transcode of unquantized tensors)
    SP_DT_F32      = 1,    // 4 bytes/elem, block_size=1
    SP_DT_F16      = 2,    // 2 bytes/elem, block_size=1
    SP_DT_BF16     = 3,    // 2 bytes/elem, block_size=1

    // PPT-native quant types
    SP_DT_OK_Q8     = 10,   // 0_K-lifted int8 + per-row Frobenius scale
                           // (scale lives in paired tensor with .scale suffix)
                           // block_size=1, 1 byte/elem
    SP_DT_OK_Q4     = 11,   // 0_K-lifted int4 packed two-per-byte
                           // block_size=32, 16 bytes per 32 elements
    SP_DT_FROBENIUS_SCALE_FP32 = 12,
                           // companion to Q8/Q4 weight tensor; one fp32 scalar
                           // per row of the weight; named "<weight>.scale"

    // Discrete-algebra-native types
    SP_DT_SPINOR63  = 20,   // 63-byte logical block, 64-byte on-disk block
                           // (see §6); block_size=64, block_count = N_blocks
    SP_DT_RING_RESIDUE_CRT_30_30 = 30,
                           // dual-prime CRT residue pair; two u32 per element
                           // block_size=8 (one (r1, r2) pair)
    SP_DT_OK_INTEGER = 31,  // 0_K[√-163] elements stored as (Re i32, Im i32)
                           // block_size=8
} sp_dtype_id;
```

`dtype_id` space is partitioned: 0-9 reserved continuous, 10-19 quant, 20-29 discrete-algebra block formats, 30-39 ring residues, 40+ reserved for v1+ formats (Stern-Brocot tables, factored ARM bank, etc.).

Each dtype's relationship between logical shape (`dims[]`) and on-disk byte length is mechanical:

dtype	logical elements per block	bytes per block on disk
F32	1	4
F16 / BF16	1	2
OK_Q8	1	1
OK_Q4	32	16
FROBENIUS_SCALE_FP32	1	4
SPINOR63	(arch-defined; one Spinor signature)	64 (63 + 1 pad)
RING_RESIDUE_CRT_30_30	1	8
OK_INTEGER	1	8

`block_size` in the tensor entry is the on-disk bytes-per-block; the loader uses `block_size * block_count == size_bytes` as a sanity invariant.

6. The Spinor 63→64 padding decision

Decision: on-disk padding to 64 bytes per Spinor block, byte 63 of each block holds the sentinel value 0xA5.

Two alternatives were considered:

Option A — on-disk padding (chosen). Each Spinor block occupies exactly 64 bytes on disk: bytes 0..62 are the Spinor payload, byte 63 is the sentinel 0xA5. The block_size field in the tensor entry is 64. The mmap'd region presents 64-byte-aligned blocks; the loader does nothing; the file IS the in-memory layout.

Option B — scatter at load. Spinor blocks are packed contiguously on disk (each 63 bytes), and the loader scatters them into a 64-byte-aligned arena at load time. Saves 1.5% of disk per Spinor tensor.

Option A wins on three structural grounds, not just preference:

1. **The ABI requires it.** PPT-LAT-L1-ABI-v0 §2 commits L1 to a caller-allocates discipline with no L1-side malloc crossing the FFI. A scatter-at-load step would have to allocate a fresh arena inside `sp_model_load`, doubling RAM during load (mmap'd source + destination arena) and adding an $O(N)$ copy. That breaks the “the file IS the load” property that justifies the format existing in the first place.
2. **Spinor blocks are read in 64-byte SIMD-friendly chunks anyway.** AVX-512 / NEON loads are 64 bytes wide. Reading a 63-byte block via `vmovdqu64` past the 63rd byte is undefined unless the byte at +63 is allocated and readable. On-disk padding makes the read well-defined; scatter-at-load forces a per-block bounds check or a slower $32+16+8+4+2+1$ fallback.
3. **The sentinel doubles as cheap integrity.** A 1.5% disk overhead is the price; in exchange, byte 63 holds 0xA5 (0b10100101 — high bit density, unlikely to arise from zero-fill or sparse-write corruption). Any page tear, partial write, or filesystem corruption that touches the Spinor block but leaves byte 63 alone is impossible by construction. The verifier (opt-in, `SP_VERIFY_TENSORS`) scans every Spinor block's byte 63 in a tight AVX-512 stride; a mismatch returns `SP_ESPINOR_BADBLOCK` per the ABI's error surface (PPT-LAT-L1-ABI §7).

Disk-overhead math: a Gemma3-1B-class model with Spinor-formatted KV adjuncts carries ~50 M Spinor blocks. At 1 byte pad each, that's 50 MB of overhead on a model whose total .sp-model size is ~2 GB. 2.5% of the Spinor portion, ~1.5% of total file size. Acceptable.

On-disk sentinel value 0xA5 rationale. Three constraints: (a) distinct from zero (catches zero-fill corruption), (b) distinct from the all-ones byte 0xFF (catches “device returned -1 on read” errors that get memcpy'd in), (c) bit pattern that's unlikely to be produced by misaligned shifts of legitimate Spinor payload bytes. 0xA5 = 0b10100101 satisfies all three.

7. Companion .sp-tokenizer file

offset	size	field	notes
0	4	magic	ASCII "SPTK"
4	2	version_major	v0 = 0
6	2	version_minor	v0 = 1
8	4	header_size	u32, v0 = 128
12	4	type_id	enum: SENTENCEPIECE=0, BPE_LLAMA3=1, BPE_GPT2=2, TIKTOKEN_0200K=3, ...
16	4	vocab_size	u32, must match .sp-model.vocab_size

20	4	bos_token	u32, token id (or 0xFFFFFFFF if absent)
24	4	eos_token	u32, token id (or 0xFFFFFFFF if absent)
28	4	pad_token	u32, token id (or 0xFFFFFFFF if absent)
32	4	unk_token	u32, token id (or 0xFFFFFFFF if absent)
36	8	blob_offset	u64, byte offset of raw tokenizer blob
44	8	blob_size	u64, length of raw tokenizer blob in bytes
52	4	header_crc32	u32, CRC-32 of bytes [0, 52)
56	72	reserved	zero-filled
128	...	blob	raw SentencePiece / BPE bytes, as-shipped

The blob is exactly what HuggingFace ships in `tokenizer.json` / `tokenizer.model` for the corresponding tokenizer type — we do not reformat or reparse it. L2 hands the blob to a SentencePiece / tokenizers crate at load time. L1 never touches it; L1 only verifies the SHA-256 of the entire `.sp-tokenizer` file matches the `tokenizer_hash` in the paired `.sp-model` header.

This is what makes `.sp-tokenizer` reusable across fine-tunes: a Llama-3-Instruct, Llama-3-Code, and Llama-3-Chat fine-tune all share the same `.sp-tokenizer`, while each ships its own `.sp-model`. The mismatch case (“you loaded a Qwen3 model with the Llama-3 tokenizer”) returns `SP_ETOKENIZER_HASH` per the ABI’s error surface.

8. Load procedure — `sp_model_load` reference implementation

In pseudocode (a real implementation is ~200 lines of C):

```
fn sp_model_load(model_path, tokenizer_path, out_model) -> sp_status:
  1. open(model_path), stat
  2. mmap(file_size, READ, SHARED, fd, 0)           // single syscall
  3. memcpy(&header, mmap_base, 512)
  4. verify header.magic == "SPMD"
  5. verify CRC32(mmap_base[0..360]) == header.header_crc32
  6. verify header.file_size == stat.st_size
  7. verify version_major == 0 (or dispatch v1 reader)
  8. memcpy(&model.arch, header.arch_struct, header.arch_struct_size)
  9. tensor_table_ptr = mmap_base + header.tensor_table_offset
 10. tensor_data_ptr = mmap_base + header.tensor_data_offset
 11. verify (header.tensor_table_offset % 64) == 0
 12. verify (header.tensor_data_offset % 65536) == 0
 13. open(tokenizer_path), stat
 14. compute SHA-256 of tokenizer file
 15. verify SHA-256 matches header.tokenizer_hash
      → on mismatch: return SP_ETOKENIZER_HASH
 16. mmap tokenizer file separately (smaller, often shared
      across many models)
 17. *out_model = sp_model {
      mmap_base, mmap_size,
      arch,
      tensor_table_ptr, tensor_count,
      tensor_data_ptr,
      tokenizer_mmap_base, tokenizer_blob_offset, tokenizer_blob_size,
    }
 18. return SP_OK
```

No malloc in the hot path; `sp_model` itself is a small heap-allocated struct holding pointers into the mmap regions. Total load time is dominated by SHA-256 of the tokenizer file (typically

<50 ms for a 1-2 MB tokenizer) plus header CRC (microseconds). The mmap is lazy — pages fault in as tensors are first accessed, which is exactly what we want.

9. gguf-to-sp transcoder responsibilities

The transcoder is the *one-shot* path from upstream GGUF to PPT- native .sp-model. Run offline, once per model, on the workstation that has the source GGUF.

Per-tensor transcoding:

- **Unquantized tensors** (norms, biases, RoPE inverse-frequency tables): copied bit-for-bit. dtype stays F32 / F16.
- **GGUF quantized tensors** (Q8_0, Q4_K, etc.): dequantize to F32, then re-quantize into 0K_Q8 or 0K_Q4 with per-row Frobenius scale. The scale becomes a separate tensor named `<original>.scale` of dtype FROBENIUS_SCALE_FP32.
- **Attention / Spinor-eligible tensors**: optionally re-pack into Spinor signatures during transcode if the source arch supports it. If not, leave as F16/F32 and let the engine apply Spinor at runtime via the KV cache hook.

Arch detection: pull `general.architecture` from the GGUF metadata, map to `sp_arch_info.arch_id`. The transcoder owns the per-arch metadata extraction (RoPE base, GQA group count, SWA window, FFN variant), populates `sp_arch_info`, embeds it into the `arch_struct` field of the header.

Tokenizer extraction: GGUF embeds the tokenizer; we strip it out and write .sp-tokenizer. SHA-256 of the resulting .sp-tokenizer goes into the .sp-model header.

Transcoder is a separate binary, sp-transcode. Not part of libshannonprime. Lives in the engine repo alongside other tools.

Spatial-locality constraint on the data-region layout. Sibling tensors **MUST** be written physically adjacent in the .sp-model data region — specifically, `<weight>.scale` immediately follows `<weight>`, with no other tensor’s bytes interposed. The transcoder sorts the data region in this order before writing:

1. Group tensors by their “base name” (everything before the final .scale, .bias, etc. suffix).
2. Within each group, write the parent first, then siblings in suffix-alphabetical order.
3. Write groups in topological order of access frequency (token embeddings first, then per-layer blocks, then output projection).

At inference time the kernel reads `weight` and immediately `weight.scale`. If they are physically separated by megabytes of unrelated tensors, mmap triggers two independent hard page faults and the OS prefetcher gets no hint that the second access is coming. If they are adjacent in the file (and therefore adjacent in the mmap region), the OS prefetcher pulls the scale page in with the weight page — a single 4 KB / 16 KB readahead window covers both. On a cold load this is the difference between ~10 µs and ~150 µs per Q8 layer’s first decode step, multiplied across every layer in the model.

The constraint is the transcoder’s responsibility; the loader does not need to validate it (the format is correct either way, just slower without the locality property). However, sp-transcode --verify should emit a warning if any sibling pair is non-adjacent, since that indicates a transcoder bug or hand-edited file.

10. Versioning and forward compatibility

- **v0** (this document): everything above.

- **v0.x** (no breaking change): new `dtype_id` values, new `arch_id` values. v0 readers refuse with `SP_EUNSUPPORTED` on unknown ids. Header bytes still occupy 512 bytes with `header_size == 512`.
- **v1** (potentially breaking): header may grow past 512 bytes. v0 readers refuse on `version_major != 0`. New fields go into the current reserved tail before growing the header.

Promotion criteria from v0 to v1: only when (a) the ABI requires a new field in `sp_arch_info` larger than the current 256-byte reservation, or (b) a structural property of the tensor data region changes (e.g. inline ARM bank initial state, KV warm-state snapshot). Neither is on the Phase-2 horizon.

11. Open questions / Phase-2+ considerations

- **Multi-file sharding for very large models.** A 200B model at Q4 is ~100 GB. A single `.sp-model` works on 64-bit filesystems but is unwieldy to distribute. v1 may add an optional shard-manifest sibling file (`.sp-shards`) pointing at multiple `.sp-model.NNNN-of-MMMM` parts. v0 assumes single file.
- **Direct DMA from .sp-model into GPU memory.** NVIDIA's GDS (GPUDirect Storage) reads from `O_DIRECT`-opened files into device memory without host buffering. Requires the tensor data region to be aligned to GPU page size (often 64 KB) and tensors to be at least 64 KB. Our existing 65536 alignment is already compatible; individual small tensors (norms, biases) below 64 KB still go through host buffering. Phase-2+ optimization.
- **In-file ARM bank seed.** Currently the ARM bank is initialized empty at session create. A pre-baked ARM bank (seeded with the golden-ratio key schedule from PPT-LAT-Systems §4.2) could ship as a regular tensor in `.sp-model`. v0 leaves the bank session-local; the spec accommodates this future addition without rev.
- **Tokenizer-blob compression.** SentencePiece blobs are usually small enough that compression is not worth it; HuggingFace `tokenizer.json` for some BPE-heavy tokenizers can hit 10 MB. v0 ships uncompressed; v1 may add a `blob_compression` field in the `.sp-tokenizer` header.

12. Sign-off checklist

Before this is locked alongside the ABI:

1. **mmap correctness.** Walk the load procedure in §8; confirm no step requires a `malloc` proportional to tensor data size. Each step is either a syscall, a `memcpy` of header-sized bytes, or a pointer assignment.
2. **Alignment table.** For every `dtype` in §5, confirm $(\text{tensor_data_offset} + \text{offset_in_data}) \% \text{required_align} == 0$ is enforceable by the transcoder. The two non-trivial cases are `SPINOR63` (`block_size` 64, so the loader can stride by 64) and `OK_Q4` (`block_size` 32, but `offset_in_data` must still be 64-aligned because we want SIMD on the first block).
3. **Round-trip.** A model transcoded `GGUF` → `.sp-model` → loaded by `sp_model_load` and run through `sp_prefill_chunk` produces logits bit-identical (deterministic mode) to running the original `GGUF` under `llama.cpp` with the same sampler off. This is the actual `T_FRO_4`-class gate for the format itself, separable from the ABI sign-off.

On green, both PPT-LAT-L1-ABI-v0 and PPT-LAT-SP-MODEL-v0 fold into PPT-LAT-Systems as Appendices A and B respectively, in one commit, and freeze together for Phase 2.

Status. v0 draft. Co-locked with PPT-LAT-L1-ABI-v0 once both have signed off against their §12 checklists.

Appendix C — Heterogeneous Compute Pipeline (HVX / HTP / ISP via Halide AOT)

The Hexagon backend is the only target in the lattice with three independent compute resources on-die: the Hexagon Vector eXtensions (HVX) on the cDSP, the Hexagon Tensor Processor (HTP, V69), and the Spectra 680 Image Signal Processor (ISP). Naive use treats them sequentially. Optimal use runs them in parallel via three composable modes — B (baseline), C (HTP-augmented), D (ISP-augmented).

This appendix documents the end-state decomposition. Phase 2-HX delivers Mode B; Modes C and D are queued as Phase 3 sub-phases (see Roadmap §11). The other three backends (CPU, CUDA, Vulkan) do not need this appendix — they have a single compute path each.

C.1 The three modes

Mode B — HVX baseline. All forward-pass compute runs on the cDSP via HVX intrinsics, dispatched from L1 via FastRPC. Frobenius arena decoded on-device, matmul + attention + FFN all on HVX. This is the minimum-viable backend that closes T_FRO_4 on the S22U. Tag lat-phase-2-hx-closed. **Status: queued (Phase 2-HX agent).**

Mode C — HTP-augmented. The heavy QK^T matmul is dispatched to the V69 HTP via QNN; FFN stays on HVX. The HTP and cDSP overlap — while HVX is computing FFN of layer N, HTP is computing QK^T of layer N+1. Engages SP_HX_MODE=C (a new session-config flag). **Status: queued (Phase 3-HX-MODE-C agent, blocked by Mode B).**

Mode D — ISP-augmented. The FFN is fused (up + gate + activation + down) at 18-bit fixed-point and dispatched to the Spectra 680 ISP via Halide AOT-compiled kernels. ISP, HTP, and HVX all run in parallel: ISP on FFN layer N, HTP on QK^T layer N+1, HVX on residual fixup + norms. Engages SP_HX_MODE=D. **Status: queued (Phase 3-HX-MODE-D agent, blocked by Mode C).**

C.2 Activation data types per mode

The activation precision floor differs by mode because each compute resource has its own native arithmetic preference:

Mode	Matmul accumulator	Activation buffer	FFN accumulator	Norm precision
B	int32 (HVX)	int16	int32 (HVX)	f32 (HVX FMA)
C	int32 (HTP)	int16	int32 (HVX)	f32 (HVX FMA)
D	int32 (HTP)	int16	int32 fixed-Q8 (ISP/Halide)	f32 (HVX FMA)

The fixed-Q8 in Mode D’s FFN accumulator means 8 fractional bits in an int32, giving 24 bits of integer headroom for the accumulator — enough margin for a 4096-wide dot product of $\text{int8} \times \text{int8}$ with the Frobenius scale applied per-row at the boundary. The Q-point scaling constants are baked into the Halide AOT pipeline at build time.

C.3 Activation function dispatch per architecture

The FFN’s nonlinearity is arch-specific. The Hexagon backend MUST dispatch on `sp_arch_info.ffn_variant` (and on `sp_arch_info.arch_id` where the variant alone is ambiguous):

arch_id	ffn_variant	Mode B/C kernel	Mode D fixed-point approximation
LLAMA3	SwiGLU	true SiLU + multiply	HardSwish-SwiGLU: $\text{up} \cdot \text{gate} \cdot \text{clamp}(\text{gate}+3, 0, 6) / 6$
QWEN3	SwiGLU	true SiLU + multiply	HardSwish-SwiGLU: $\text{up} \cdot \text{gate} \cdot \text{clamp}(\text{gate}+3, 0, 6) / 6$
GEMMA3	GeGLU	GELU-tanh + multiply	Piecewise polynomial GeGLU (see §C.4)
DEEPSEEK	SwiGLU	true SiLU + multiply	HardSwish-SwiGLU

The HardSwish-SwiGLU formula must include the $\cdot \text{gate} / 6$ term. A common early-implementation bug is to write $\text{up} \cdot \text{clamp}(\text{gate}+3, 0, 6)$ and call it HardSwish; that's actually $\text{up} \cdot 6 \cdot \text{HardSigmoid}(\text{gate})$, which is numerically very different and tanks PPL. The correct formula is $\text{up} \cdot \text{gate} \cdot \text{clamp}(\text{gate}+3, 0, 6) / 6$. Every Mode D implementation MUST verify against the f32 SwiGLU oracle before shipping ($\text{gate_E_HX_D_SWIGLU_KL} \leq 2\text{e-}3$ on Qwen3-0.6B chunked prefill).

C.4 GeGLU piecewise polynomial approximation (Gemma3 only)

True GELU on the DSP would emit emulated $\exp / \tanh$ and stall the HVX pipeline. The fixed-point Mode D path uses the standard tanh approximation:

$$\text{gelu}(x) \approx 0.5 \cdot x \cdot (1 + \tanh_{\text{approx}}(0.7978 \cdot (x + 0.044715 \cdot x^3)))$$

where $\tanh_{\text{approx}}(y)$ is a 5th-order odd polynomial fit over $y \in [-2.5, 2.5]$ with hard-clamp outside that range:

$$\tanh_{\text{approx}}(y) = \text{clamp}(y, -2.5, 2.5) \cdot (1 - (y^2 \cdot (a_1 + y^2 \cdot (a_2 + y^2 \cdot a_3))))$$

with the coefficients a_1, a_2, a_3 fit by least-squares against exact tanh on the clamped range. The gate is $\text{E_HX_D_GEGLU_KL} \leq 2\text{e-}3$ on Gemma3-1B chunked prefill against the f32 GELU-tanh oracle. The actual coefficients are baked into the Halide AOT pipeline; the Mode D agent will publish them in the closure SESSION-STATE.

C.5 sp_session_config.thermal_pause_us — config knob

Hexagon-specific: the S22U thermal-soaks in 30-60 seconds of sustained ISP + HTP + HVX parallel operation, after which the firmware throttles cDSP throughput by ~40%. A 1-2 ms pause between layers lets the thermal sensor ride the limit without triggering the throttle.

This is a session-config knob, not hardcoded:

```
typedef struct {
    size_t    max_context;
    bool      deterministic;
    uint32_t  arm_bank_kb;
    uint32_t  sieve_capacity;
    uint32_t  flags;
    uint32_t  thermal_pause_us; // NEW: per-layer pause in microseconds
                                // 0 = none (CPU/CUDA/Vulkan default)
                                // 1500 = Hexagon Mode D default
                                // ignored when deterministic == true
} sp_session_config;
```

`thermal_pause_us` slots into the reserved tail of `sp_session_config` introduced in PPT-LAT-L1-ABI-v0 §6, so this is a non-breaking ABI addition — v0 binaries reading the new field get a zero (matching old behavior on non-Hexagon backends). Determinism mode forces the value to 0 because wall-clock affects nothing about the math; the pauses exist only for thermal management.

The Mode D agent will profile to determine the optimal default per target hardware. The S22U baseline is 1500 μ s; tablets with active cooling can drop to 0; fanned dev kits (Snapdragon X Elite reference boards) likewise 0.

C.6 The Halide AOT compilation pipeline

Halide is a build-time tool, not a runtime dependency. The Mode D agent runs the Halide generator on the Windows host to emit architecture-specific static archives:

```
generate_ffn_skeleton.exe -g ffn_skeleton -e static_library,h \
-o . target=hexagon-v69-no_asserts
```

This emits `ffn_skeleton.a` (HVX object code) and `ffn_skeleton.h` (the C ABI). The `.a` is linked into `libffn_fusion_skel.so` on the DSP side via `hexagon-clang` (the Hexagon SDK's compiler, NOT the Android NDK).

Per-arch dispatch is handled at Halide generator time, not at runtime: the agent emits `ffn_skeleton_llama3.a`, `ffn_skeleton_gemma3.a`, etc. The L1 backend selects the right archive by `arch_id` at link time, not via a runtime switch. This keeps the hot path branch-free and lets each archive carry only its own activation polynomial.

C.7 FastRPC / SVM lifecycle

Per the L1 ABI's no-malloc-on-hot-path rule, all `rpcmem_alloc` calls happen inside `sp_session_create` on the Hexagon backend, sized against `sp_session_config.max_context`. The zero-copy ION-heap pointers are stashed in the opaque `sp_session` struct and reused step-by-step in `sp_prefill_chunk` and `sp_decode_step`. `sp_session_destroy` calls `rpcmem_free` on every pointer it owns.

Activation scales (per-row Frobenius f32) are pre-converted to int32 Q-point at session create and stored alongside the ION buffers, so the hot path never crosses the float $\rightarrow$ fixed boundary. Passing raw `float*` scales over FastRPC every layer would emulated-cast on the DSP at significant latency cost — prohibited by this appendix.

The IDL contract uses `rout` (not `inout`) for buffers that the DSP only writes; this saves the copy-in of uninitialized output state on every call. The full IDL is in `MODE_D_DESIGN_DRAFT.md`.

C.8 Mode-D-specific error codes

Two new status codes slot into the existing `sp_status` enum from PPT-LAT-L1-ABI-v0 §7 in the backend-specific block:

```
SP_EHX_ISP_DISPATCH = -44,    // ISP failed to dispatch the Halide kernel
                             // (typically: ADSP_LIBRARY_PATH unset or
                             // libffn_fusion_skel.so missing on device)
SP_EHX_THERMAL_TRIP  = -45,    // firmware throttled while in flight;
                             // L2 should increase thermal_pause_us and retry
```

`SP_EHX_THERMAL_TRIP` is a soft error — L2's correct response is to bump the session's `thermal_pause_us` by 500 μ s and reissue, not to fail the request. This is the same recovery pattern as `SP_ECONTEXT_FULL` (eviction-then-retry).

C.9 Anti-contamination check for Mode D implementation

The proposal that this appendix codifies originates partly in the old cohort's work — specifically D:\F\shannon-prime-repos\ shannon-prime-engine (the freethedsp/ backend, the backends/halide/ build driver) and the prior phone-running ISP-fusion experiments. Per the project's anti-contamination rule, those artifacts CANNOT be copied into the lattice cohort:

- **Semantic** patterns carry forward: yes use Halide AOT, yes bind rpcmem allocation to session lifecycle, yes pause 1500 μ s between layers on S22U, yes set the ADSP_LIBRARY_PATH trailing semicolon.
- **Code** must be reimplemented fresh inside shannon-prime-system-engine/src/backends/hexagon/ and the associated Halide generators.

Both the Mode C and Mode D agents will be briefed on this boundary explicitly when they spawn.